

Introduction to Big Data

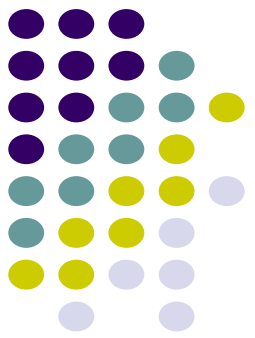
Material adapted from slides by Jimmy Lin, Christophe Bisciglia, Aaron Kimball, & Sierra Michels-Slettvet, Google Distributed Computing Seminar, 2007 (licensed under Creative Commons Attribution 3.0 License)

This work is licensed under a Creative Commons Attribution-Noncommercial-Share Alike 3.0 United States
See <http://creativecommons.org/licenses/by-nc-sa/3.0/us/> for details

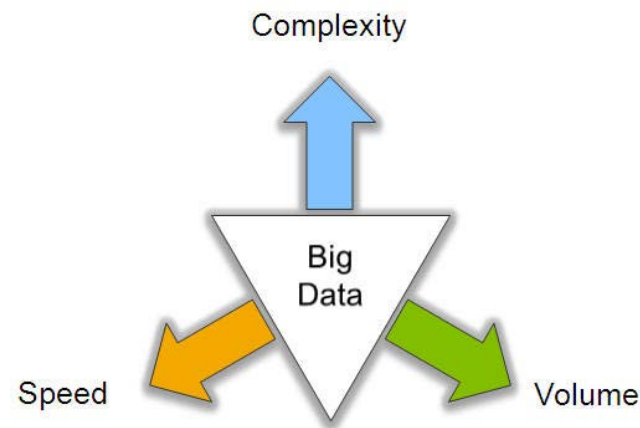
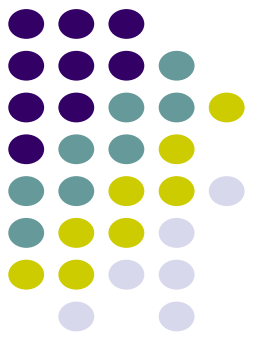
What's Big Data?

No single definition; here is from Wikipedia:

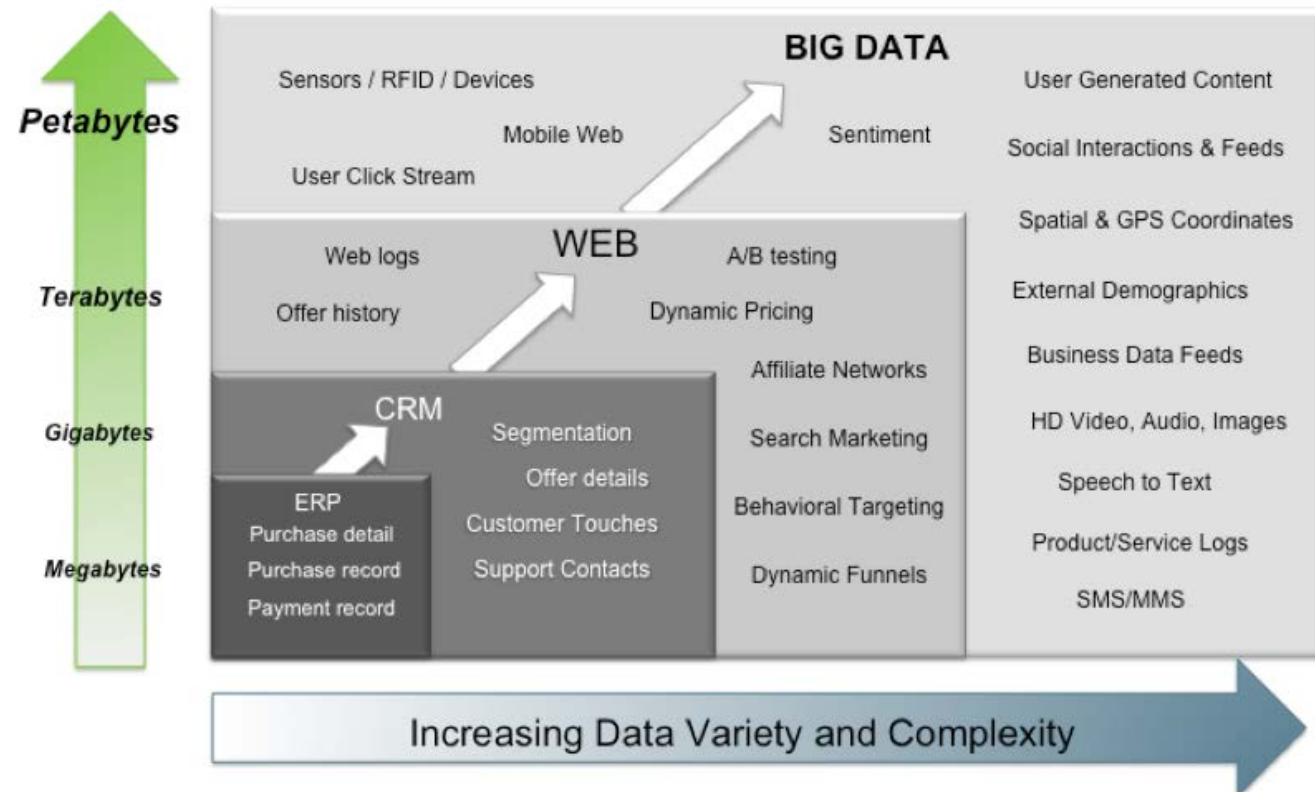
- **Big data** is the term for a collection of data sets so large and complex that it becomes difficult to process using on-hand database management tools or traditional data processing applications.
- The challenges include capture, curation, storage, search, sharing, transfer, analysis, and visualization.
- The trend to larger data sets is due to the additional information derivable from analysis of a single large set of related data, as compared to separate smaller sets with the same total amount of data, allowing correlations to be found to "spot business trends, determine quality of research, prevent diseases, link legal citations, combat crime, and determine real-time roadway traffic conditions."



Big Data: 3V's

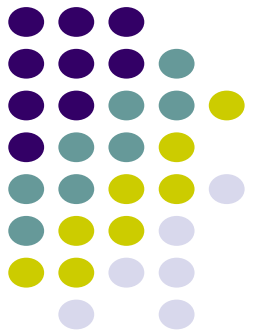


Big Data = Transactions + Interactions + Observations

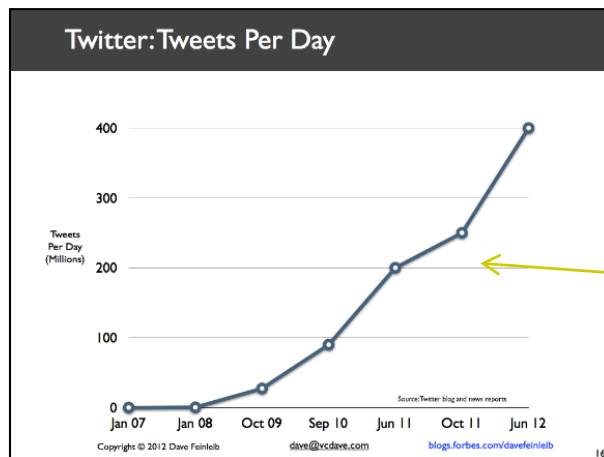
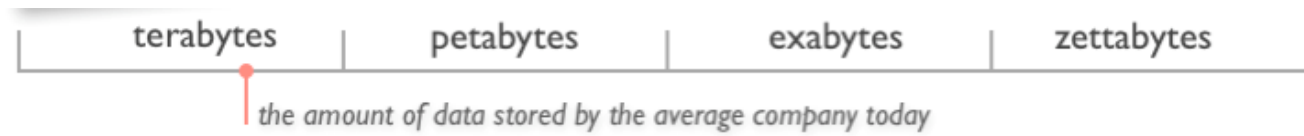


Source: Contents of above graphic created in partnership with Teradata, Inc.

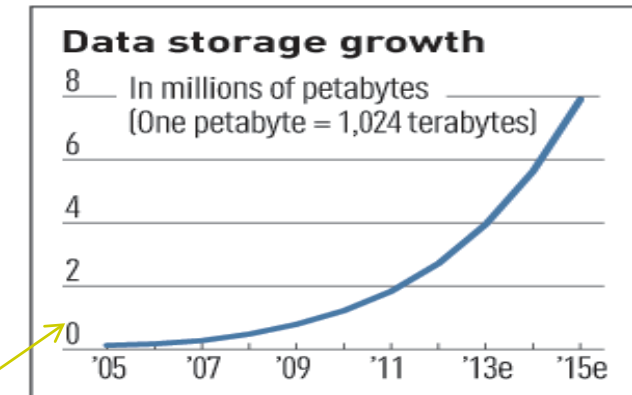
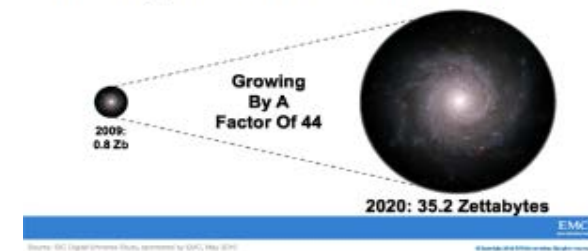
Volume (Scale)



- **Data Volume**
 - 44x increase from 2009 to 2020
 - From 0.8 zettabytes to 35.2 zb
- Data volume is increasing exponentially



The Digital Universe 2009-2020



Exponential increase in collected/generated data

? TBs of
data every day

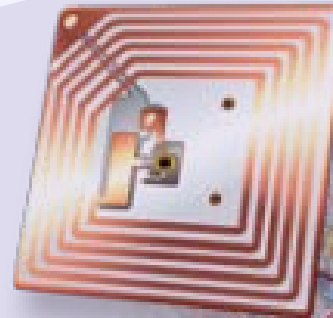


12+ TBs
of tweet data
every day



25+ TBs of
log data
every day

30 billion RFID
tags today
(1.3B in 2005)



76 million smart
meters in 2009...
200M by 2014



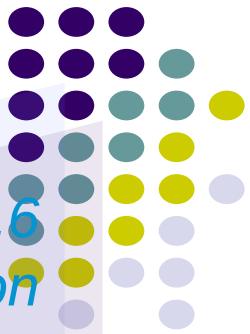
4.6 billion
camera
phones
world wide



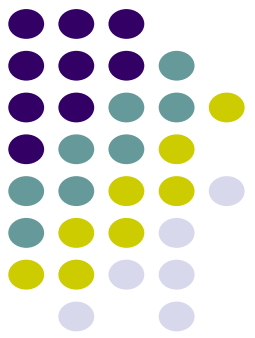
100s of millions
of GPS
enabled
devices
sold
annually

http://www.

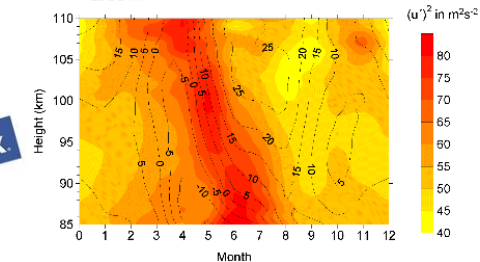
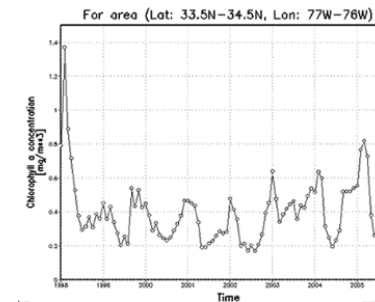
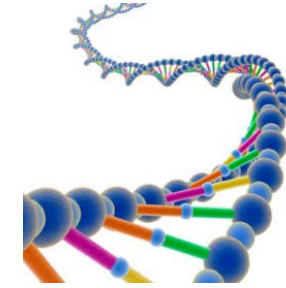
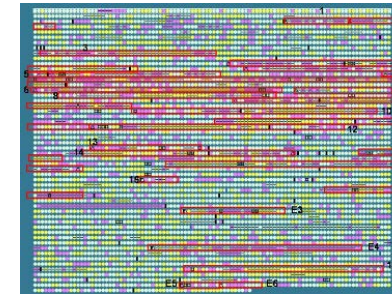
2+ billion
people on
the Web
by end
2011



Variety (Complexity)

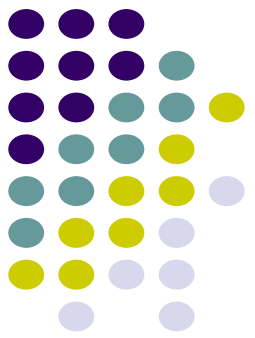
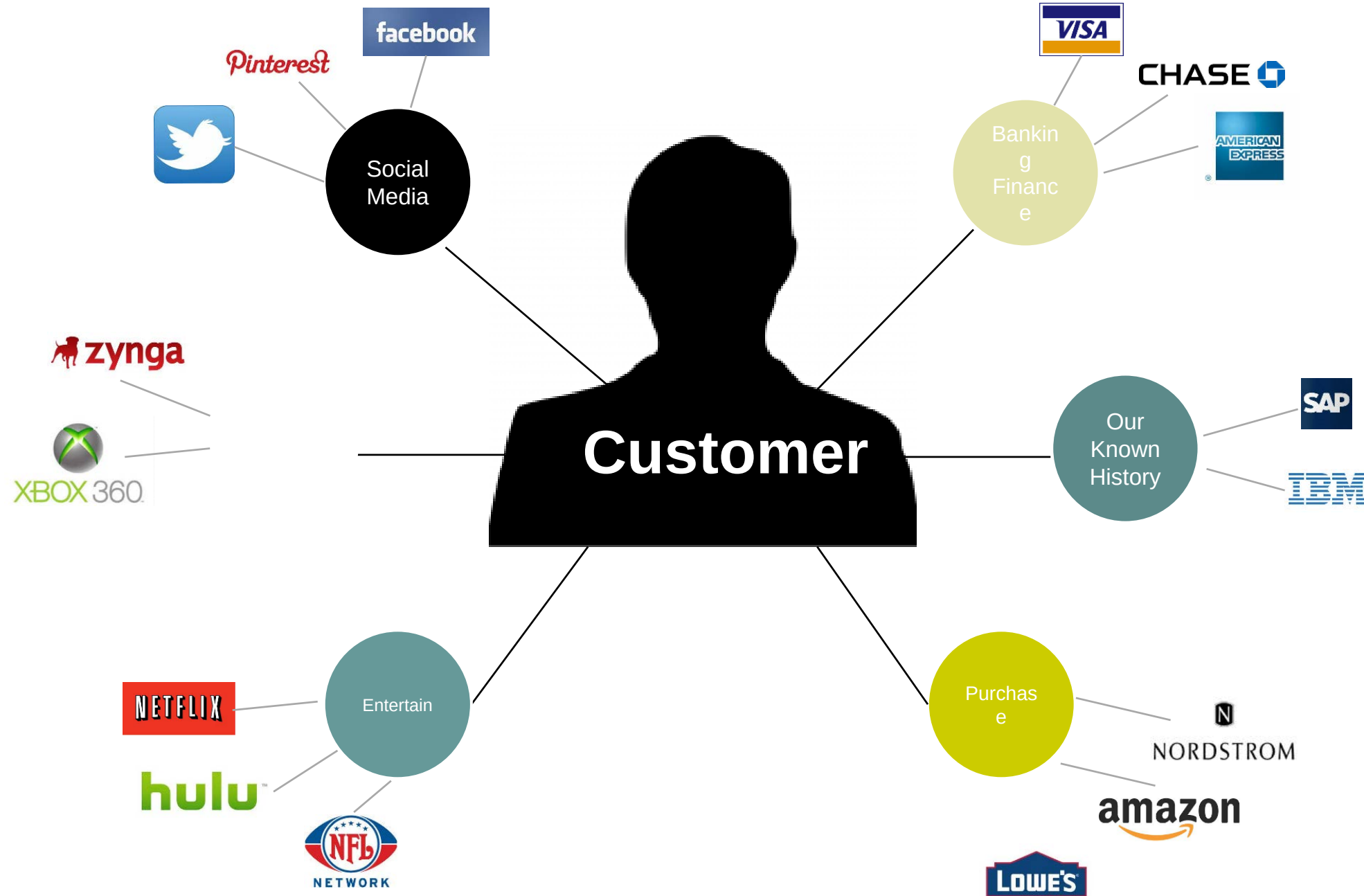


- Relational Data (Tables/Transaction/Legacy Data)
- Text Data (Web)
- Semi-structured Data (XML)
- Graph Data
 - Social Network, Semantic Web (RDF), ...
- Streaming Data
 - You can only scan the data once
- A single application can be generating/collecting many types of data
- Big Public Data (online, weather, finance, etc)

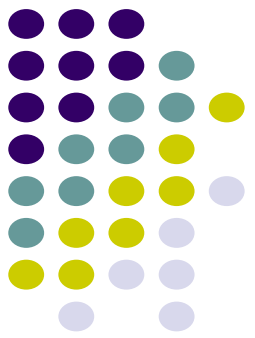


To extract knowledge → all these types of data need to be linked together

A Single View to the Customer



Velocity (Speed)



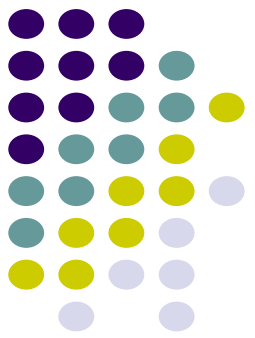
- Data is begin generated fast and need to be processed fast
- Online Data Analytics
- Late decisions → missing opportunities



- **Examples**

- **E-Promotions:** Based on your current location, your purchase history, what you like → send promotions right now for store next to you
- **Healthcare monitoring:** sensors monitoring your activities and body → any abnormal measurements require immediate reaction

Real-time/Fast Data



Social media and networks
(all of us are generating data)



Scientific instruments
(collecting all sorts of data)



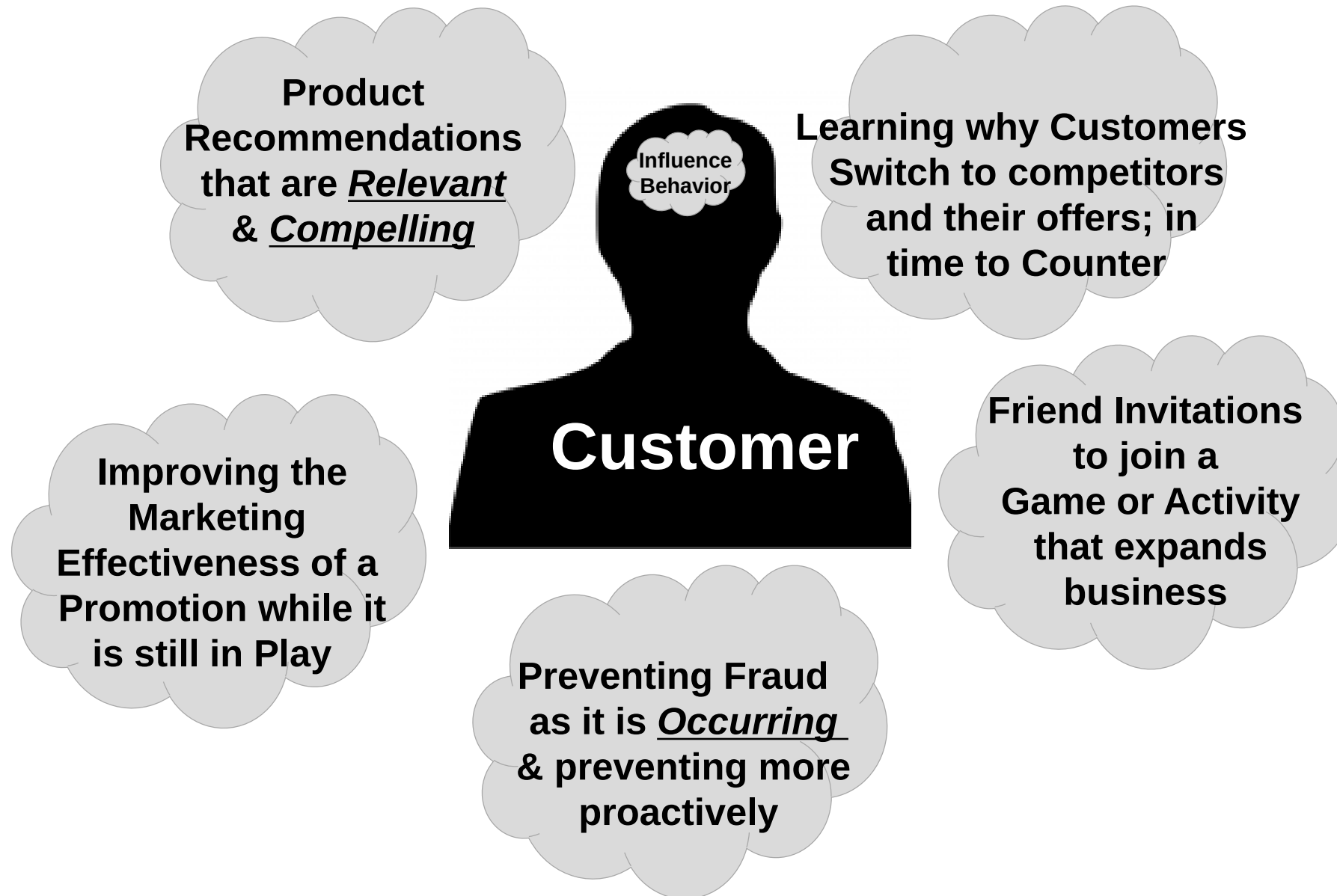
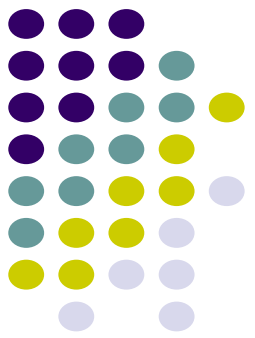
Mobile devices
(tracking all objects all the time)



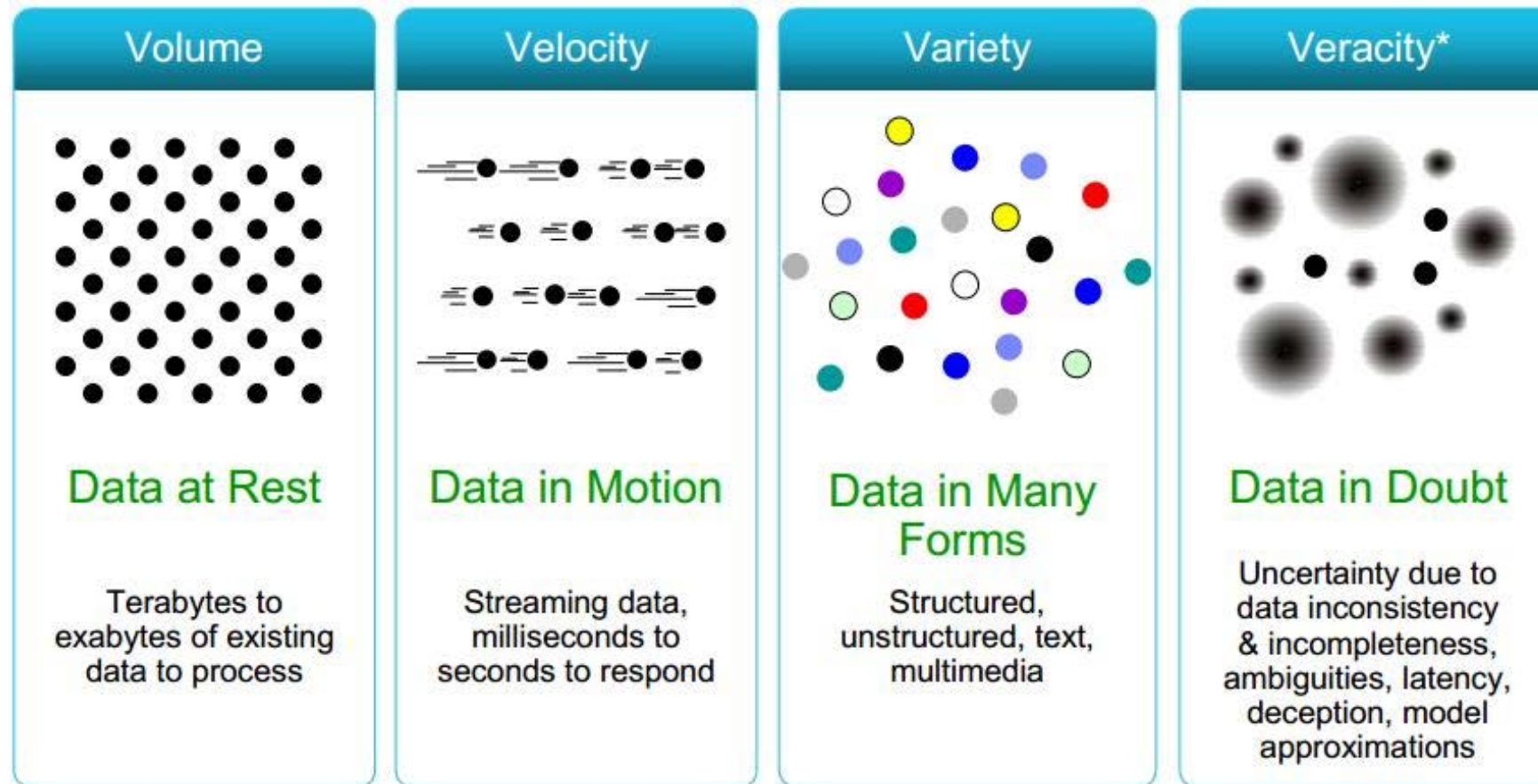
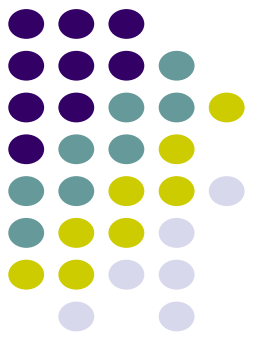
Sensor technology and networks
(measuring all kinds of data)

- The progress and innovation is no longer hindered by the ability to collect data
- But, by the ability to manage, analyze, summarize, visualize, and discover knowledge from the collected data in a timely manner and in a scalable fashion

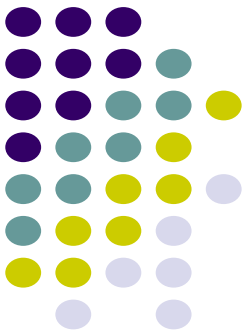
Real-Time Analytics/Decision Requirement



Some Make it 4V's



Introduction to Hadoop



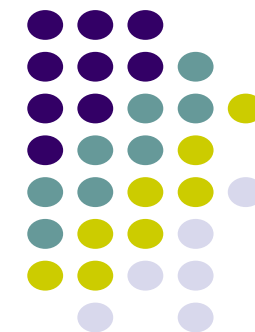
Open-Source Solution for Huge Data Sets

Courtesy: Zheng Shao

Hadoop Committer - Apache Software Foundation

11/23/2008

Data Scalability Problems



- **Search Engine**

- 10KB / doc * 20B docs = 200TB
- Reindex every 30 days: 200TB/30days = 6 TB/day

- **Log Processing / Data Warehousing**

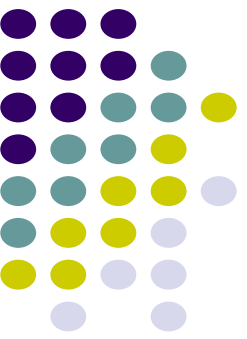
- 0.5KB/events * 3B pageview events/day = 1.5TB/day
- 100M users * 5 events * 100 feed/event * 0.1KB/feed = 5TB/day

- **Multipliers:** 3 copies of data, 3-10 passes of raw data

- **Processing Speed (Single Machine)**

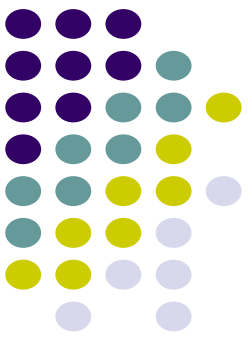
- 2-20MB/second * 100K seconds/day = 0.2-2 TB/day

Google's Solution



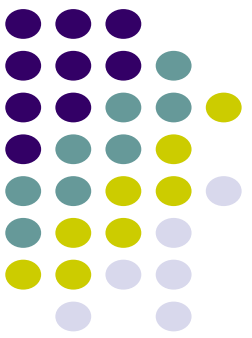
- **Google File System** – *SOSP'2003*
- **Map-Reduce** – *OSDI'2004*
- **Sawzall** – *Scientific Programming Journal'2005*
- **Big Table** – *OSDI'2006*
- **Chubby** – *OSDI'2006*

Open Source World's Solution



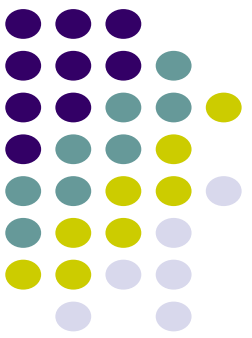
- **Google File System** – *Hadoop Distributed FS*
- **Map-Reduce** – *Hadoop Map-Reduce*
- **Sawzall** – *Pig, Hive, JAQL*
- **Big Table** – *Hadoop HBase, Cassandra*
- **Chubby** – *Zookeeper*

Hadoop History



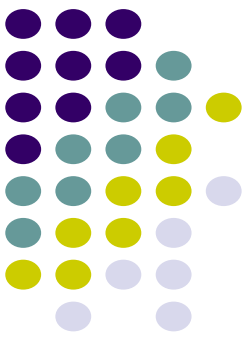
- **Jan 2006 – Doug Cutting joins Yahoo**
- Feb 2006 – Hadoop splits out of Nutch and Yahoo starts using it.
- **Dec 2006 – Yahoo creating 100-node Webmap with Hadoop**
- Apr 2007 – Yahoo on 1000-node cluster
- Jan 2008 – Hadoop made a top-level Apache project
- **Dec 2007 – Yahoo creating 1000-node Webmap with Hadoop**
- Sep 2008 – Hive added to Hadoop as a contrib project

Hadoop Introduction

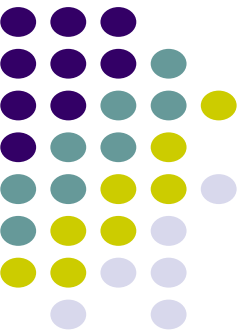


- **Open Source Apache Project**
 - <http://hadoop.apache.org/>
 - Book: <http://oreilly.com/catalog/9780596521998/index.html>
- **Written in Java**
 - Does work with other languages
- **Runs on**
 - Linux, Windows and more
 - Commodity hardware with high failure rate

Hadoop Components

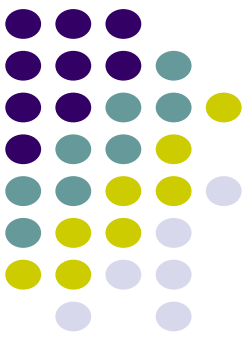


- **Hadoop Distributed File System (HDFS)**
- **Hadoop Map-Reduce**
- **Contributes**
 - Hadoop Streaming
 - Pig / JAQL / Hive
 - HBase
 - Hama / Mahout



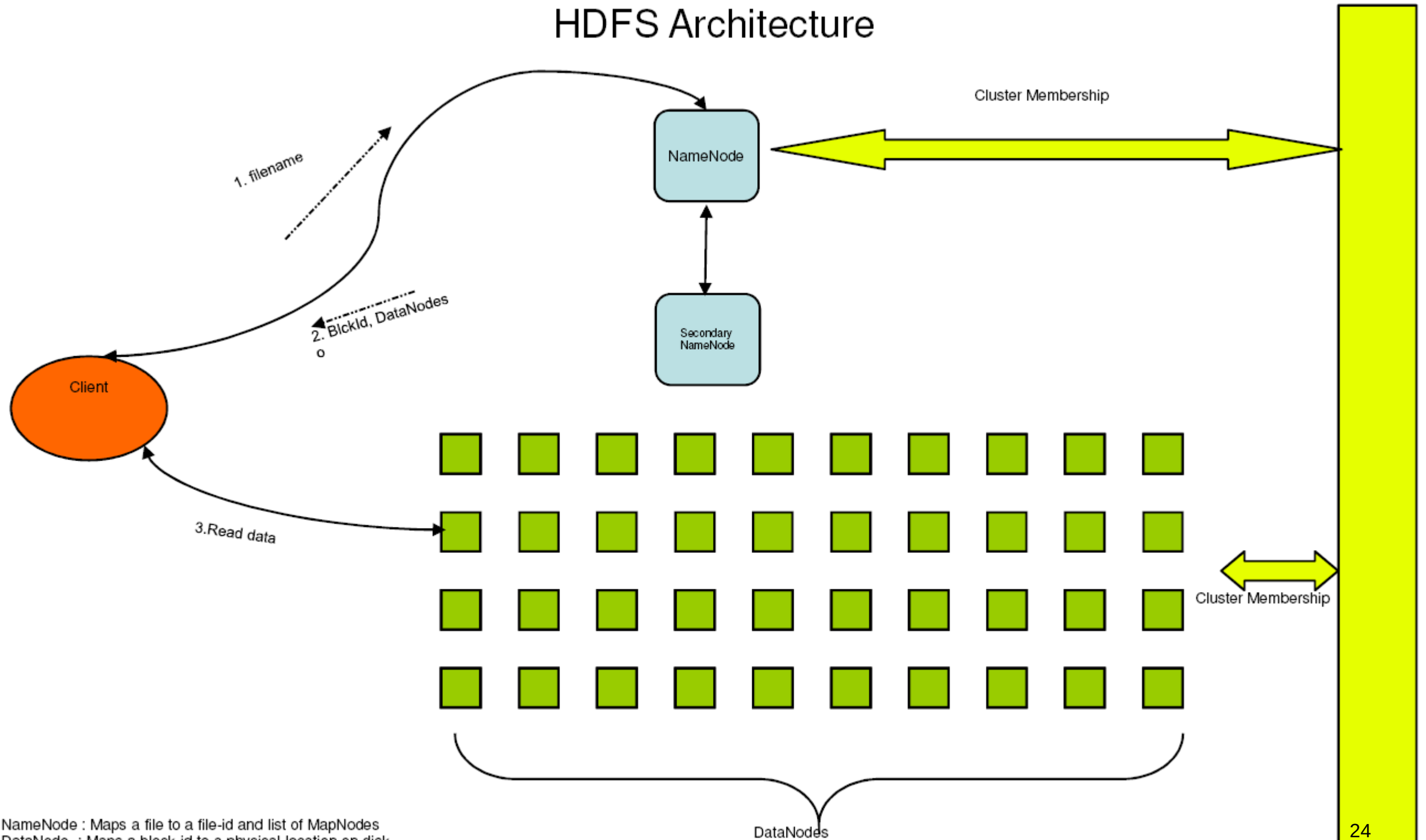
Hadoop Distributed File System

Goals of HDFS

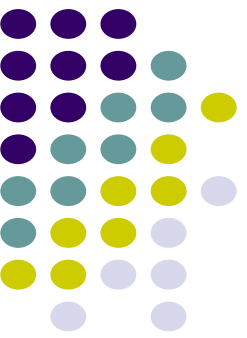


- **Very Large Distributed File System**
 - 10K nodes, 100 million files, 10 PB
- **Convenient Cluster Management**
 - Load balancing
 - Node failures
 - Cluster expansion
- **Optimized for Batch Processing**
 - Allow move computation to data
 - Maximize throughput

HDFS Architecture



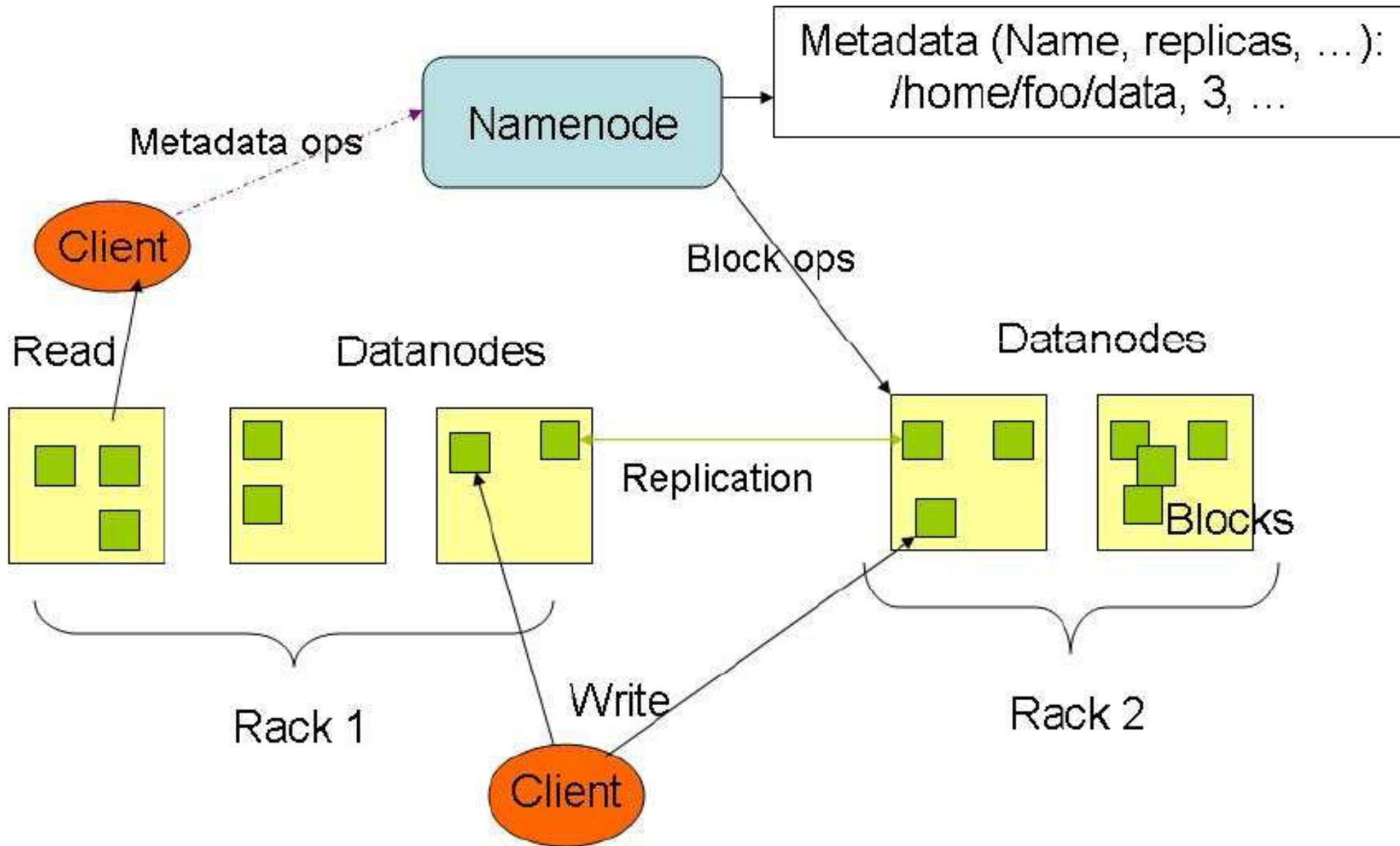
NameNode : Maps a file to a file-id and list of MapNodes
DataNode : Maps a block-id to a physical location on disk
SecondaryNameNode: Periodic merge of Transaction log



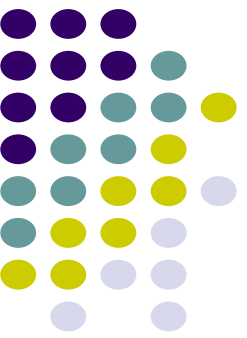
HDFS Details

- **Data Coherency**
 - Write-once-read-many access model
 - Client can only append to existing files
- **Files are broken up into blocks**
 - Typically 128 MB block size
 - Each block replicated on multiple DataNodes
- **Intelligent Client**
 - Client can find location of blocks
 - Client accesses data directly from DataNode

HDFS Architecture

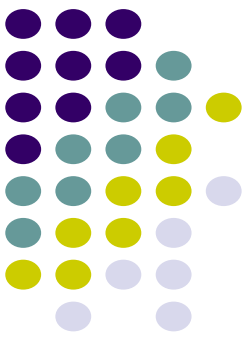


HDFS User Interface



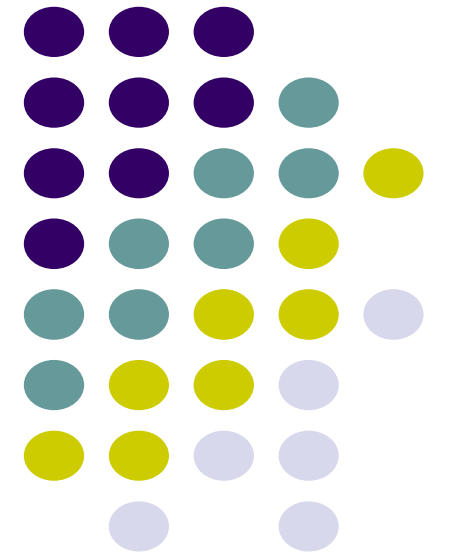
- **Java API**
- **Command Line**
 - `hadoop dfs -mkdir /foodir`
 - `hadoop dfs -cat /foodir/myfile.txt`
 - `hadoop dfs -rm /foodir myfile.txt`
 - `hadoop dfsadmin -report`
 - `hadoop dfsadmin -decommission datanodename`
- **Web Interface**
 - `http://host:port/dfshealth.jsp`

More about HDFS



- http://hadoop.apache.org/core/docs/current/hdfs_design.html
- Hadoop FileSystem API
 - HDFS
 - Local File System
 - Kosmos File System (KFS)
 - Amazon S3 File System

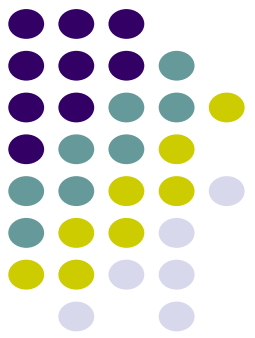
Introduction to MapReduce



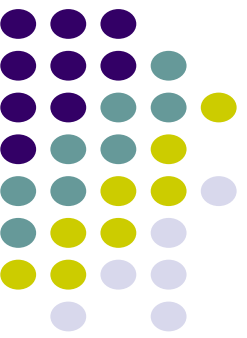
Material adapted from slides by Jimmy Lin, Christophe Bisciglia, Aaron Kimball, & Sierra Michels-Slettvet, Google Distributed Computing Seminar, 2007 (licensed under Creative Commons Attribution 3.0 License)

This work is licensed under a Creative Commons Attribution-Noncommercial-Share Alike 3.0 United States License. See <http://creativecommons.org/licenses/by-nc-sa/3.0/us/> for details

Functional Programming



- MapReduce = functional programming meets distributed processing on steroids
 - Not a new idea... dates back to the 50's (or even 30's)
- What is functional programming?
 - Computation as application of functions
 - Theoretical foundation provided by lambda calculus
- How is it different?
 - Traditional notions of “data” and “instructions” are not applicable
 - Data flows are implicit in program
 - Different orders of execution are possible
- Exemplified by LISP and ML



Overview of Lisp

- Lisp ≠ Lost In Silly Parentheses
- We'll focus on particular a dialect: “Scheme”
- Lists are primitive data types
- Functions written in prefix notation

`'(1 2 3 4 5)`

`'((a 1) (b 2) (c 3))`

`(+ 1 2) → 3`

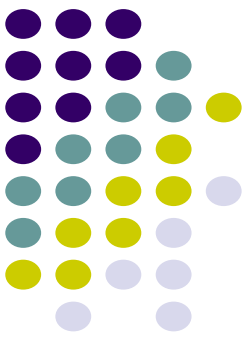
`(* 3 4) → 12`

`(sqrt (+ (* 3 3) (* 4 4))) → 5`

`(define x 3) → x`

`(* x 5) → 15`

Functions



- Functions = lambda expressions bound to variables

```
(define foo  
  (lambda (x y)  
    (sqrt (+ (* x x) (* y y)))))
```

- Syntactic sugar for defining functions

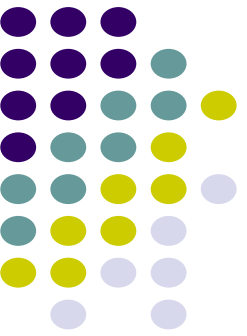
- Above expressions is equivalent to:

```
(define (foo x y)  
  (sqrt (+ (* x x) (* y y))))
```

- Once defined, function can be applied:

```
(foo 3 4) → 5
```


Other Features



- In Scheme, everything is an s-expression
 - No distinction between “data” and “code”
 - Easy to write self-modifying code
- Higher-order functions
 - Functions that take other functions as arguments

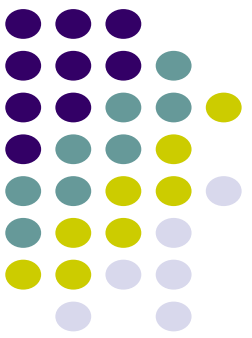
```
(define (bar f x) (f (f x)))
```

Doesn't matter what *f* is, just apply it twice.

```
(define (baz x) (* x x))
```

```
(bar baz 2) → 16
```

Recursion is your friend



- Simple factorial example

```
(define (factorial n)
  (if (= n 1)
      1
      (* n (factorial (- n 1)))))
```

(factorial 6) → 720

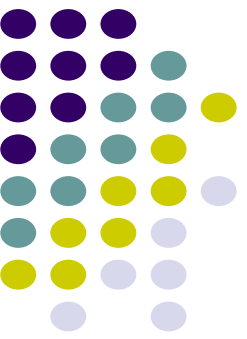
- Even iteration is written with recursive calls!

```
(define (factorial-iter n)
  (define (aux n top product)
    (if (= n top)
        (* n product)
        (aux (+ n 1) top (* n product)))))
```

(aux 1 n 1))

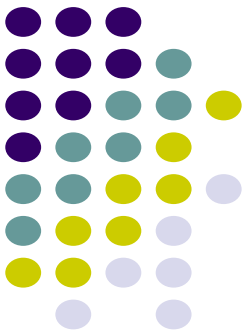
(factorial-iter 6) → 720

Lisp → MapReduce?

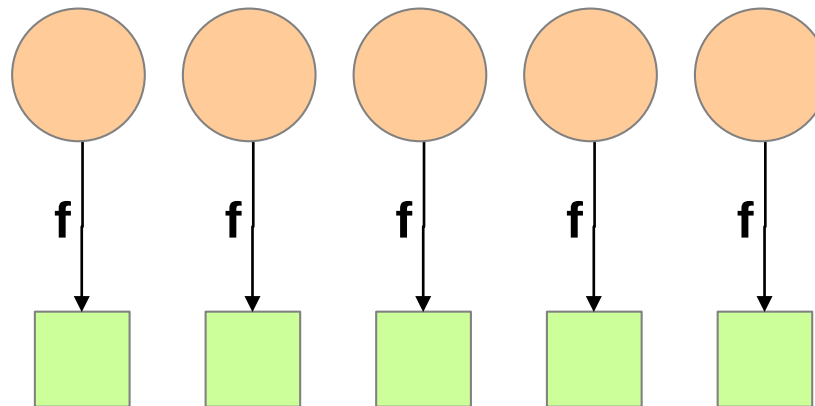


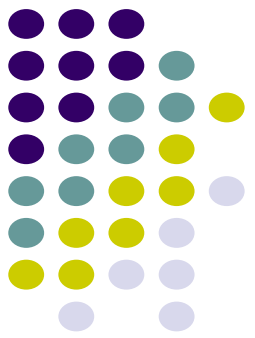
- What does this have to do with MapReduce?
- After all, Lisp is about processing *lists*
- Two important concepts in functional programming
 - Map: do something to everything in a list
 - Fold: combine results of a list in some way

Map



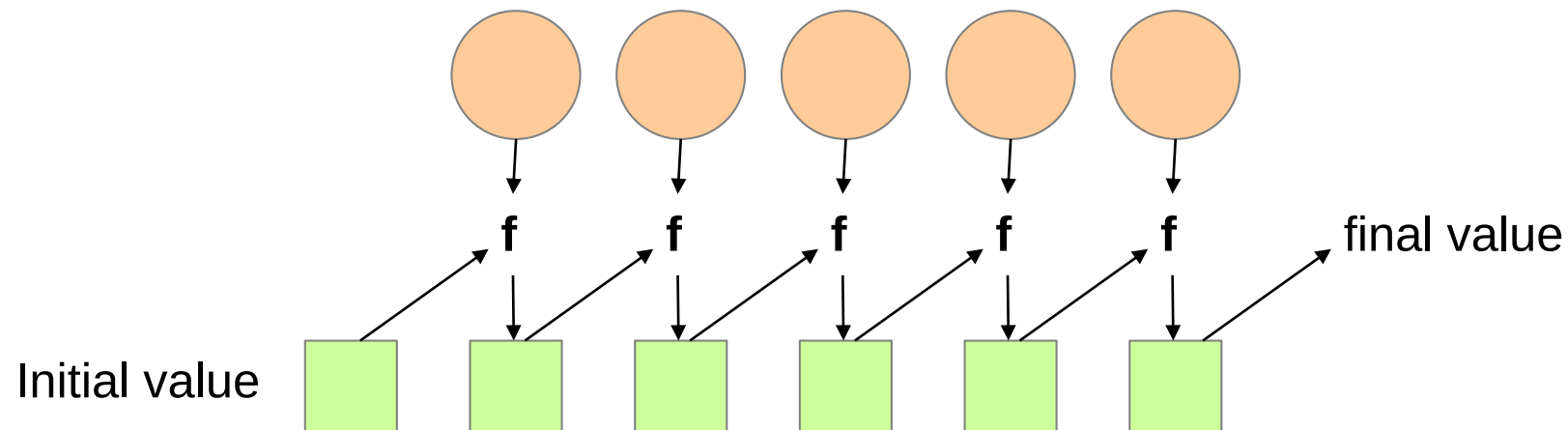
- Map is a higher-order function
- How map works:
 - Function is applied to every element in a list
 - Result is a new list

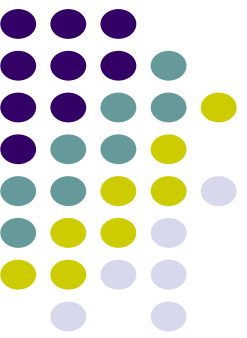




Fold

- Fold is also a higher-order function
- How fold works:
 - Accumulator set to initial value
 - Function applied to list element and the accumulator
 - Result stored in the accumulator
 - Repeated for every item in the list
 - Result is the final value in the accumulator





Map/Fold in Action

- Simple map example:

```
(map (lambda (x) (* x x))  
      '(1 2 3 4 5))  
→ '(1 4 9 16 25)
```

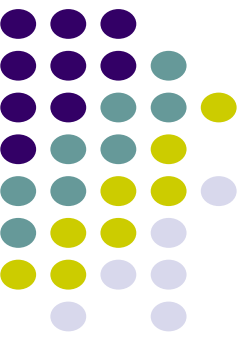
- Fold examples:

```
(fold + 0 '(1 2 3 4 5)) → 15  
(fold * 1 '(1 2 3 4 5)) → 120
```

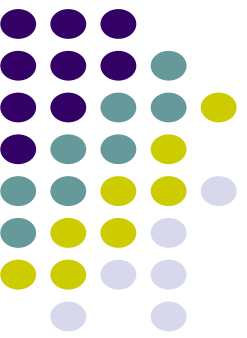
- Sum of squares:

```
(define (sum-of-squares v)  
  (fold + 0 (map (lambda (x) (* x x)) v)))  
(sum-of-squares '(1 2 3 4 5)) → 55
```

Lisp → MapReduce



- Let's assume a long list of records: imagine if...
 - We can parallelize map operations
 - We have a mechanism for bringing map results back together in the fold operation
- That's MapReduce! (and Hadoop)
- Observations:
 - No limit to map parallelization since maps are independent
 - We can reorder folding if the fold function is commutative and associative

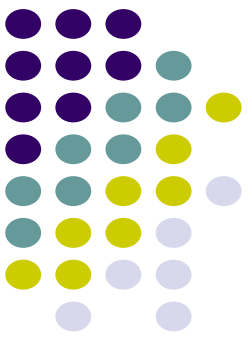


Typical Problem

- **Map** Iterate over a large number of records
- Extract something of interest from each
- Shuffle and sort intermediate **Reduce** results
- Aggregate intermediate results
- Generate final output

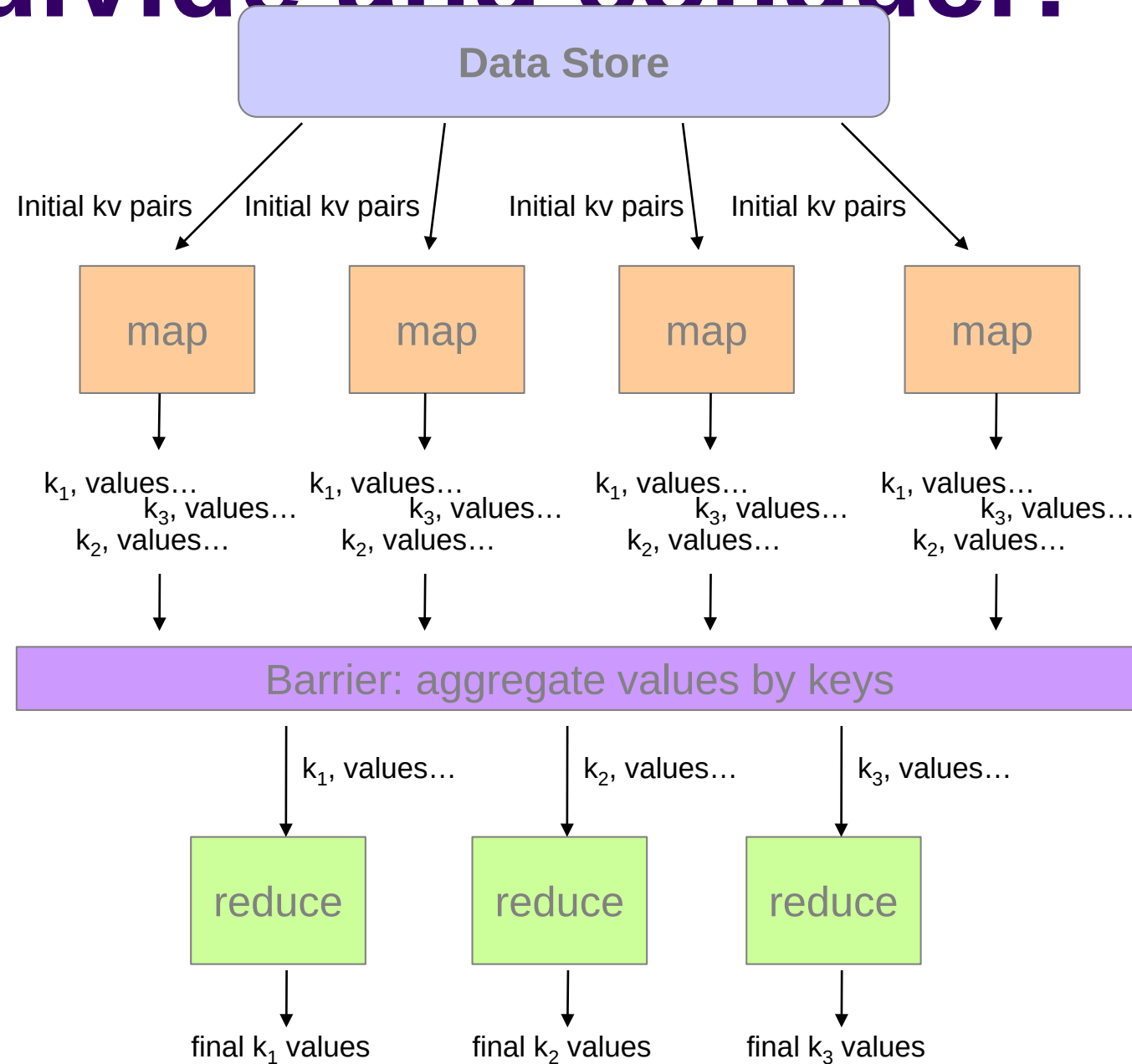
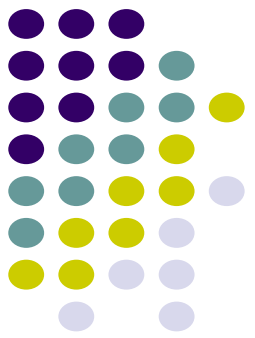
Key idea: provide an abstraction at the point of these two operations

MapReduce

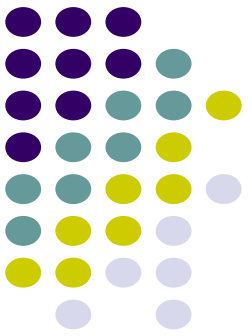


- Programmers specify two functions:
map $(k, v) \rightarrow \langle k', v' \rangle^*$
reduce $(k', v') \rightarrow \langle k', v' \rangle^*$
 - All v' with the same k' are reduced together
- Usually, programmers also specify:
partition $(k', \text{number of partitions}) \rightarrow \text{partition for } k'$
 - Often a simple hash of the key, e.g. $\text{hash}(k') \bmod n$
 - Allows reduce operations for different keys in parallel
- Implementations:
 - Google has a proprietary implementation in C++
 - Hadoop is an open source implementation in Java (lead by Yahoo)

It's just divide and conquer!



“Hello World”: Word Count



```
Map(String input_key, String input_value):
```

```
    // input_key: document name
```

```
    // input_value: document contents
```

```
    for each word w in input_values:
```

```
        EmitIntermediate(w, "1");
```

```
Reduce(String key, Iterator intermediate_values):
```

```
    // key: a word, same for input and output
```

```
    // intermediate_values: a list of counts
```

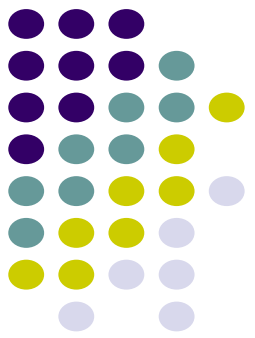
```
    int result = 0;
```

```
    for each v in intermediate_values:
```

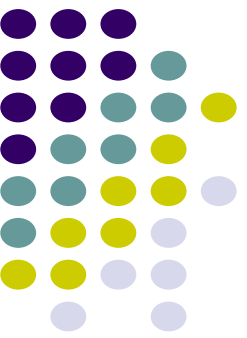
```
        result += ParseInt(v);
```

```
    Emit(AsString(result));
```

Word Count over a Given Set of Web Pages



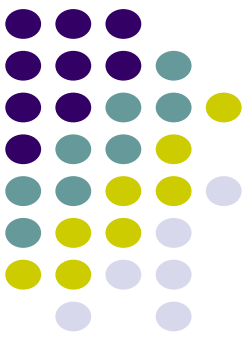
Can we do word count in parallel?



Recall these problems?

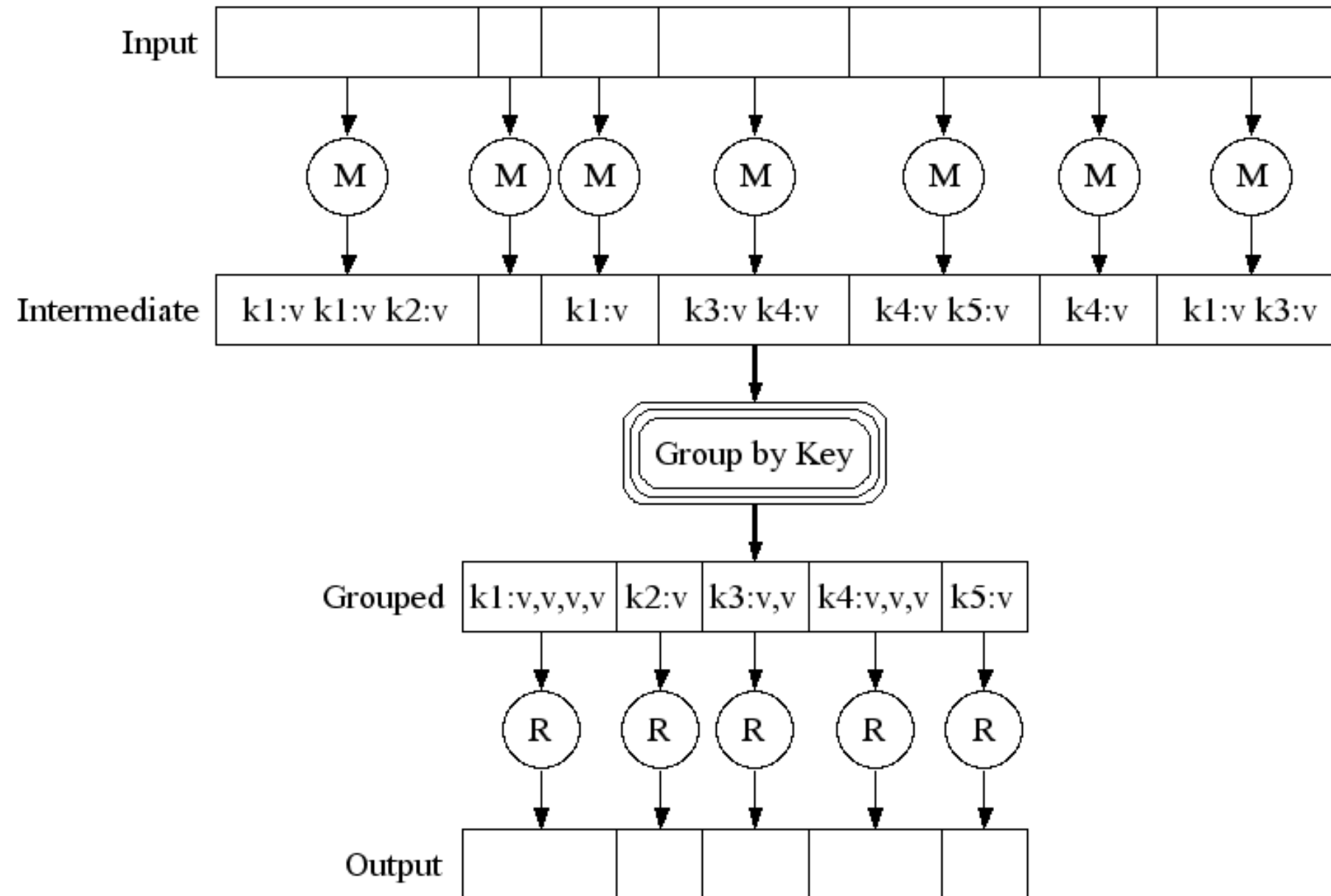
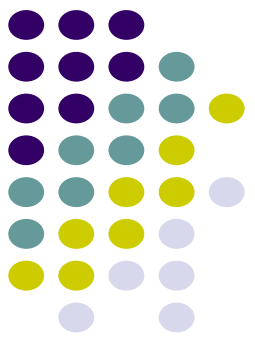
- How do we assign work units to workers?
- What if we have more work units than workers?
- What if workers need to share partial results?
- How do we aggregate partial results?
- How do we know all the workers have finished?
- What if workers die?

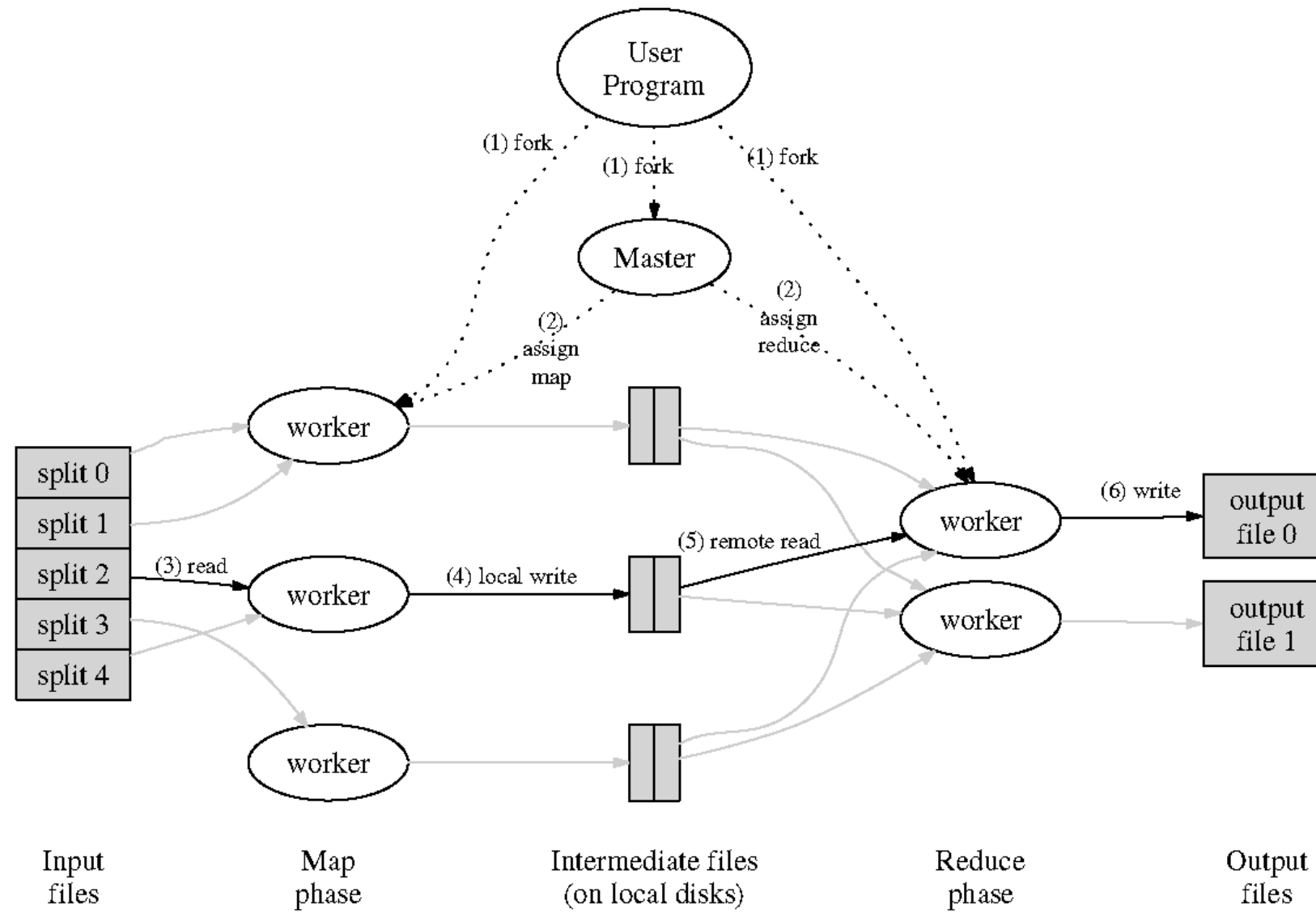
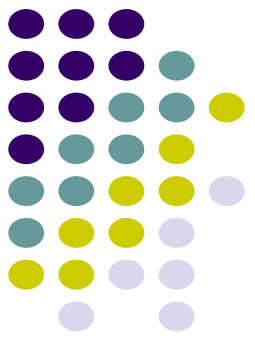
MapReduce Runtime



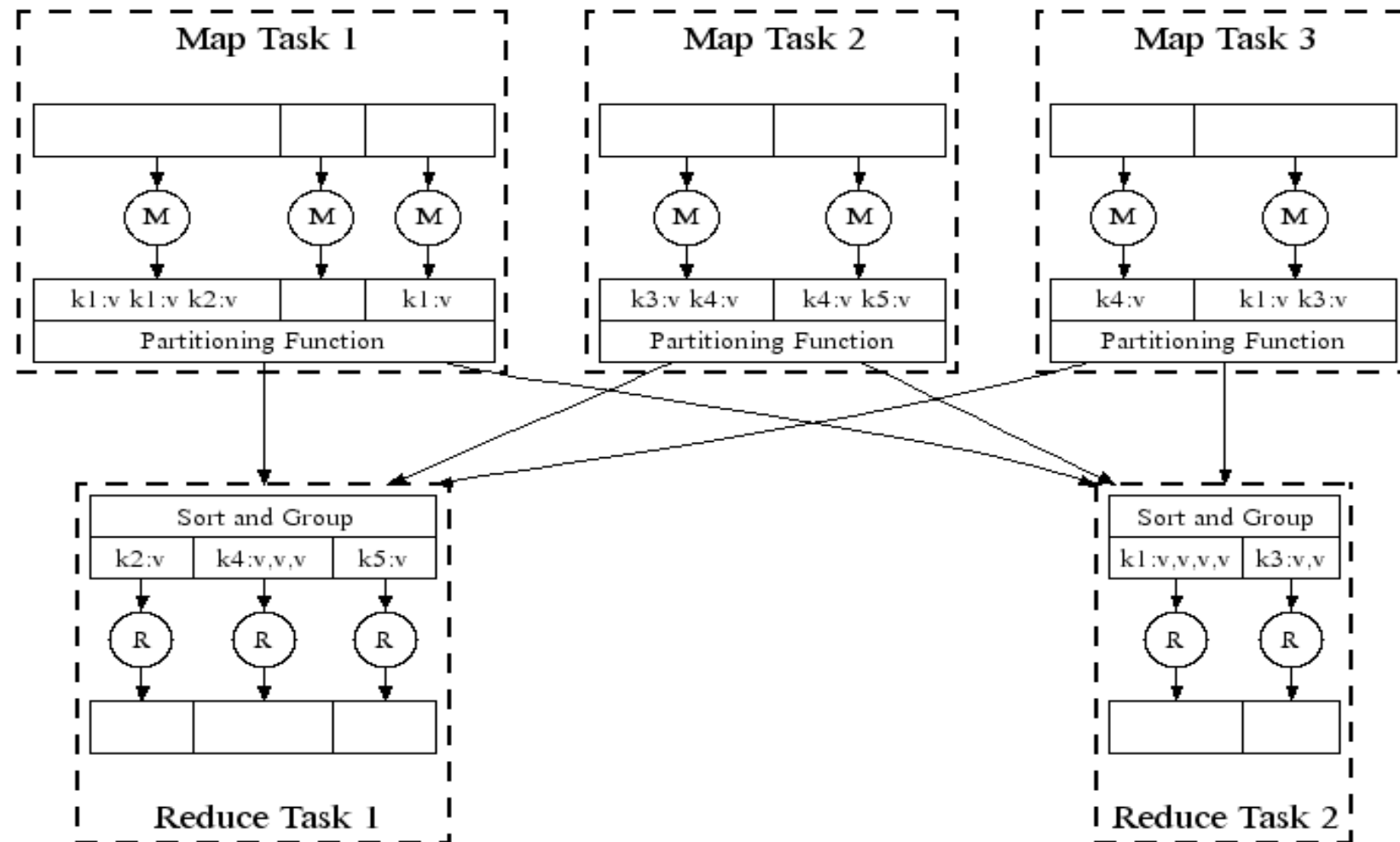
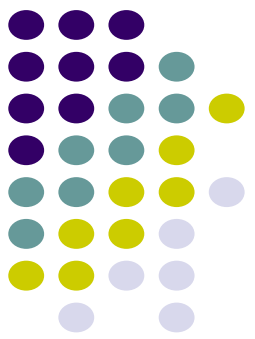
- Handles scheduling
 - Assigns workers to map and reduce tasks
- Handles “data distribution”
 - Moves the process to the data
- Handles synchronization
 - Gathers, sorts, and shuffles intermediate data
- Handles faults
 - Detects worker failures and restarts
- Everything happens on top of a distributed FS (later)

The MapReduce Framework (pioneered by Google)





Automatic Parallel Execution in MapReduce (Google)



Handles failures automatically, e.g., restarts tasks if a node fails; runs multiples copies of the same task to avoid a slow task slowing down the whole job

MapReduce in Hadoop (1)

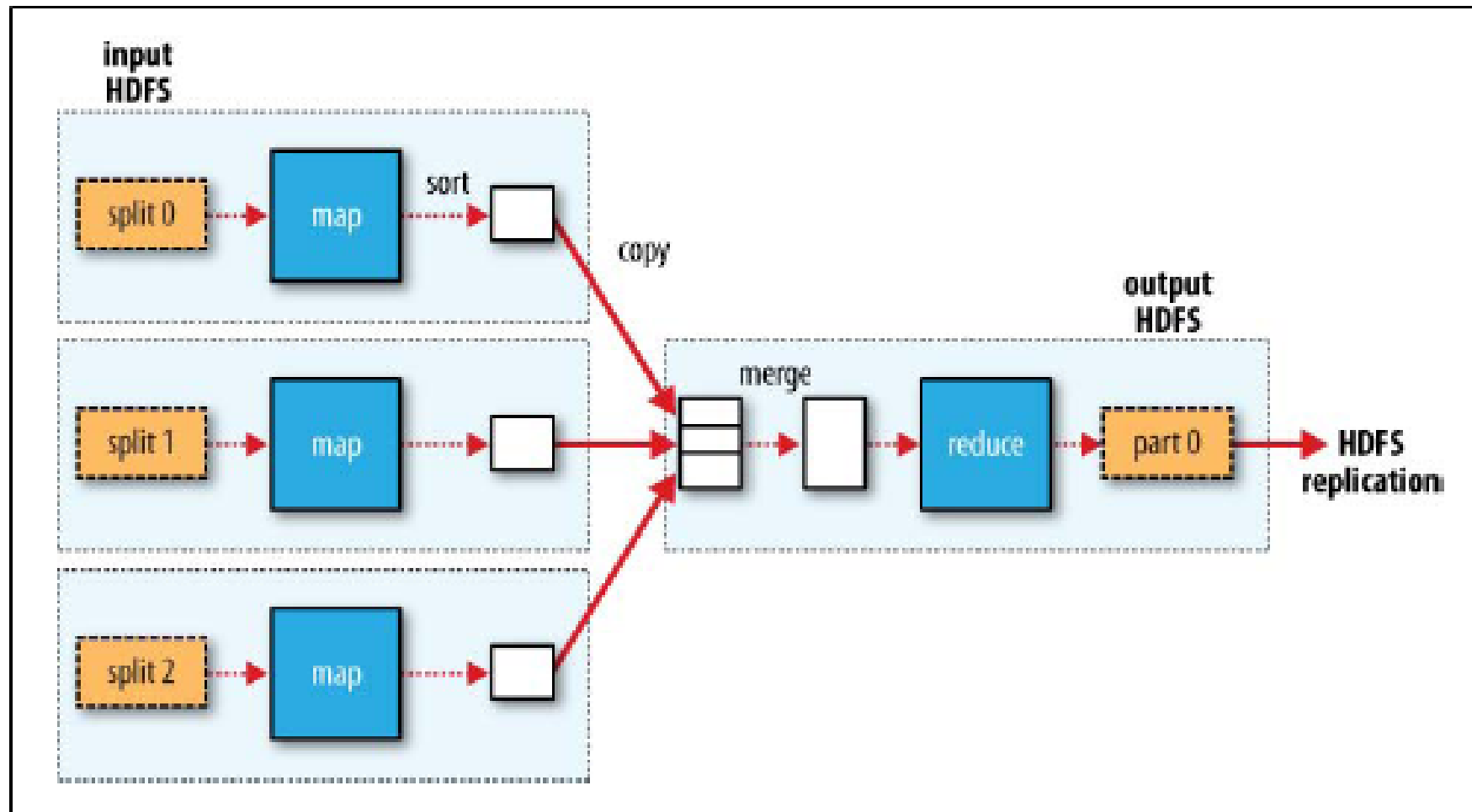
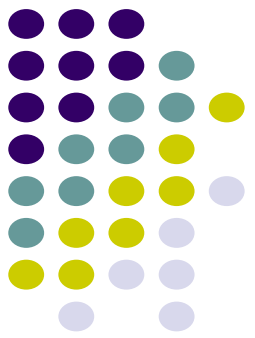


Figure 2-2. MapReduce data flow with a single reduce task

MapReduce in Hadoop (2)

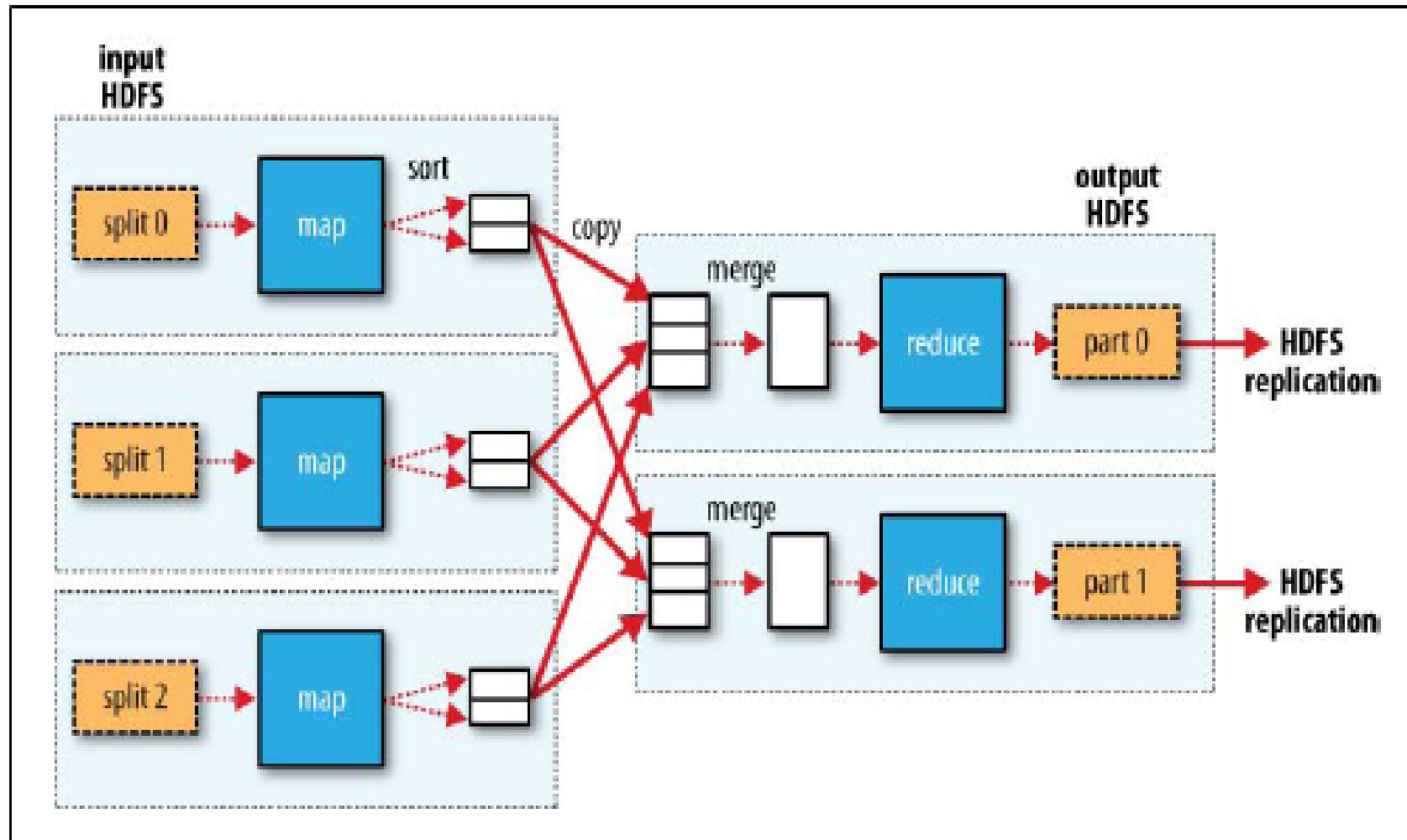
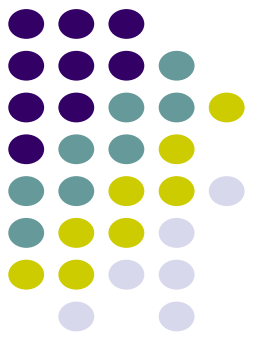


Figure 2-3. MapReduce data flow with multiple reduce tasks

MapReduce in Hadoop (3)

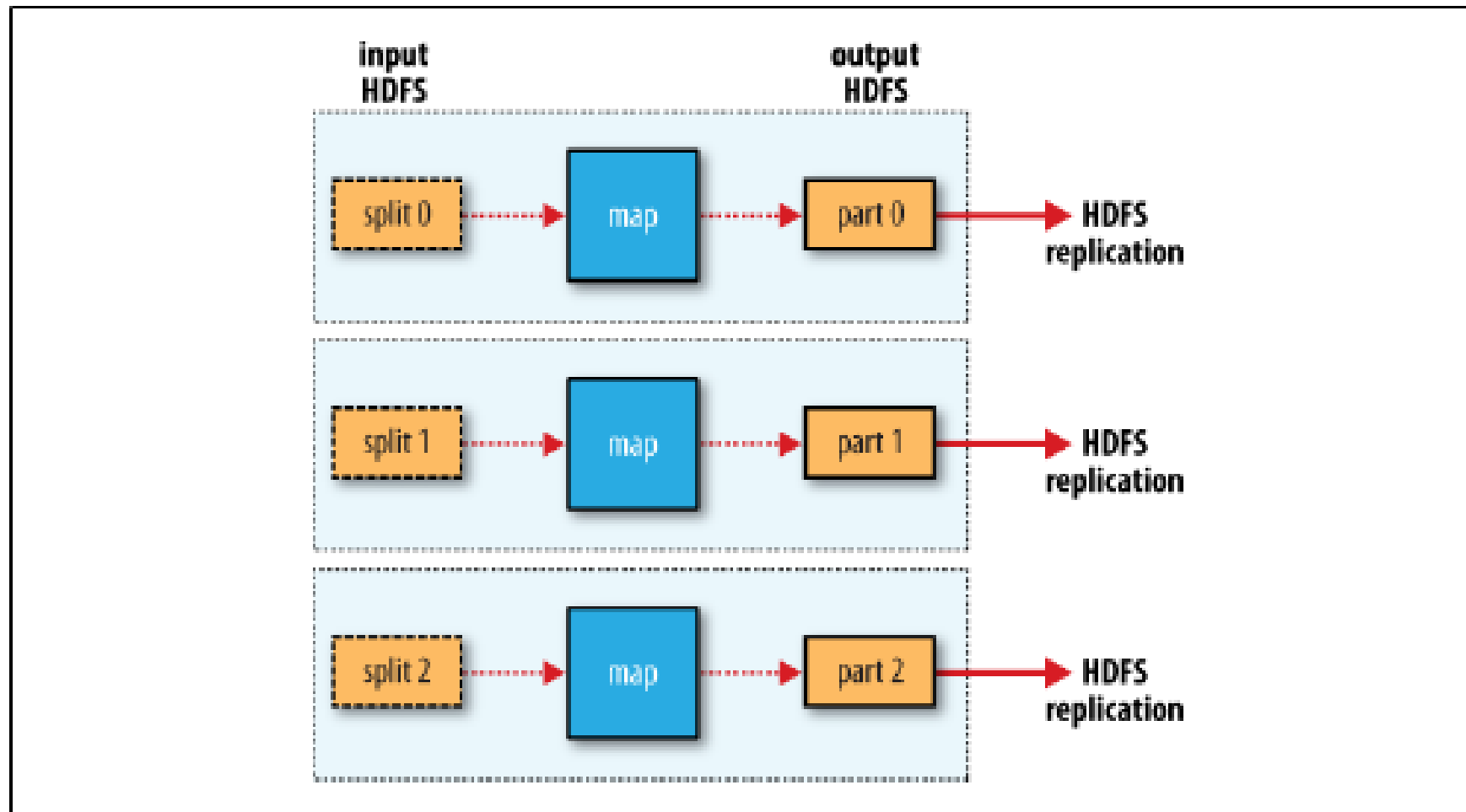
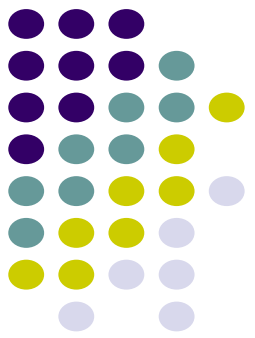
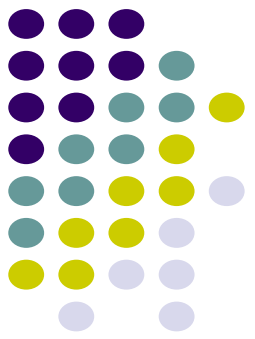
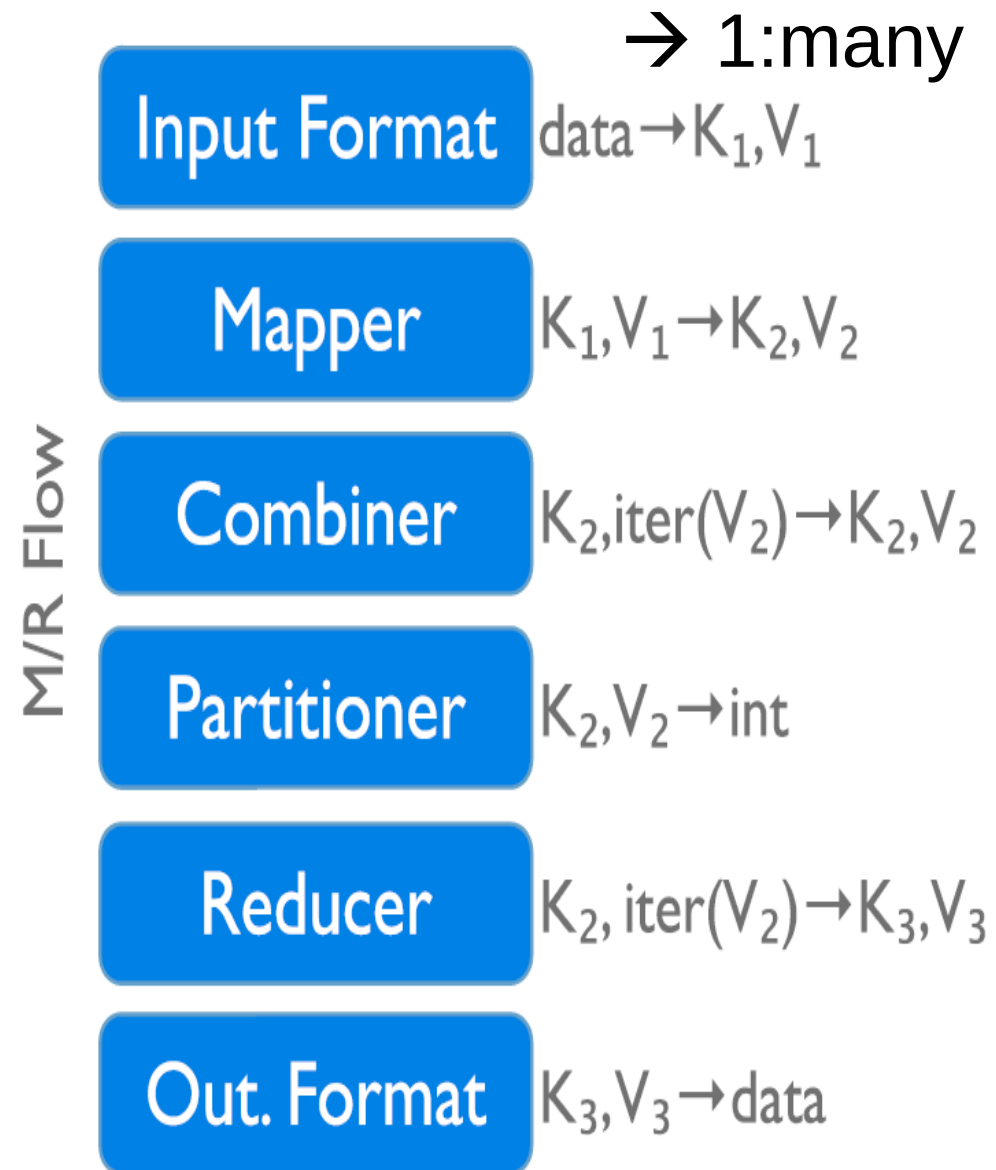


Figure 2-4. MapReduce data flow with no reduce tasks

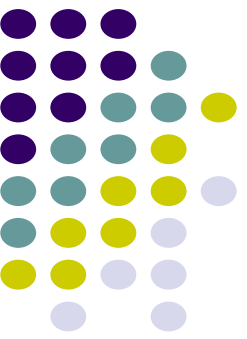
Data Flow in a MapReduce Program in Hadoop



- InputFormat
- Map function
- Partitioner
- Sorting & Merging
- Combiner
- Shuffling
- Merging
- Reduce function
- OutputFormat

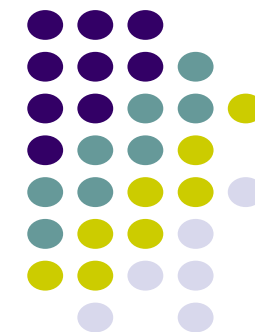


Hadoop Streaming



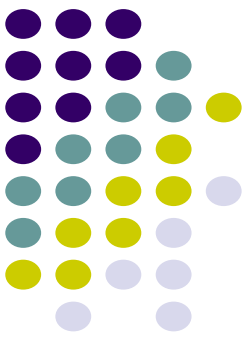
- **Allow to write Map and Reduce functions in any languages**
 - Hadoop Map/Reduce only accepts Java
- **Example: Word Count**
 - `hadoop streaming`
 - `-input /user/zshao/articles`
 - `-mapper 'tr " " "\n"'`
 - `-reducer 'uniq -c'`
 - `-output /user/zshao/`
 - `-numReduceTasks 32`

Example: Log Processing

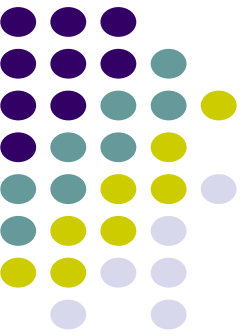


- **Generate #pageview and #distinct users for each page each day**
 - Input: timestamp url userid
- **Generate the number of page views**
 - Map: emit `< <date(timestamp), url>, 1>`
 - Reduce: add up the values for each row
- **Generate the number of distinct users**
 - Map: emit `< <date(timestamp), url, userid>, 1>`
 - Reduce: For the set of rows with the same `<date(timestamp), url>`, count the number of distinct users by `"uniq -c"`

Example: Page Rank

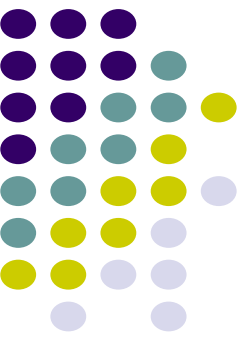


- **In each Map/Reduce Job:**
 - **Map:** emit `<link, eigenvalue(url)/#links>`
for each input: `<url, <eigenvalue, vector<link>> >`
 - **Reduce:** add all values up for each link, to generate the new eigenvalue for that link.
- Run 50 map/reduce jobs till the eigenvalues are stable.



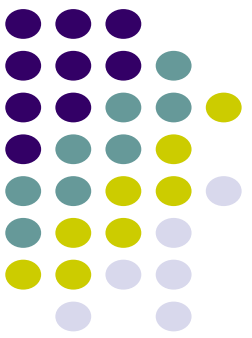
Hive - SQL on top of Hadoop

Map-Reduce and SQL



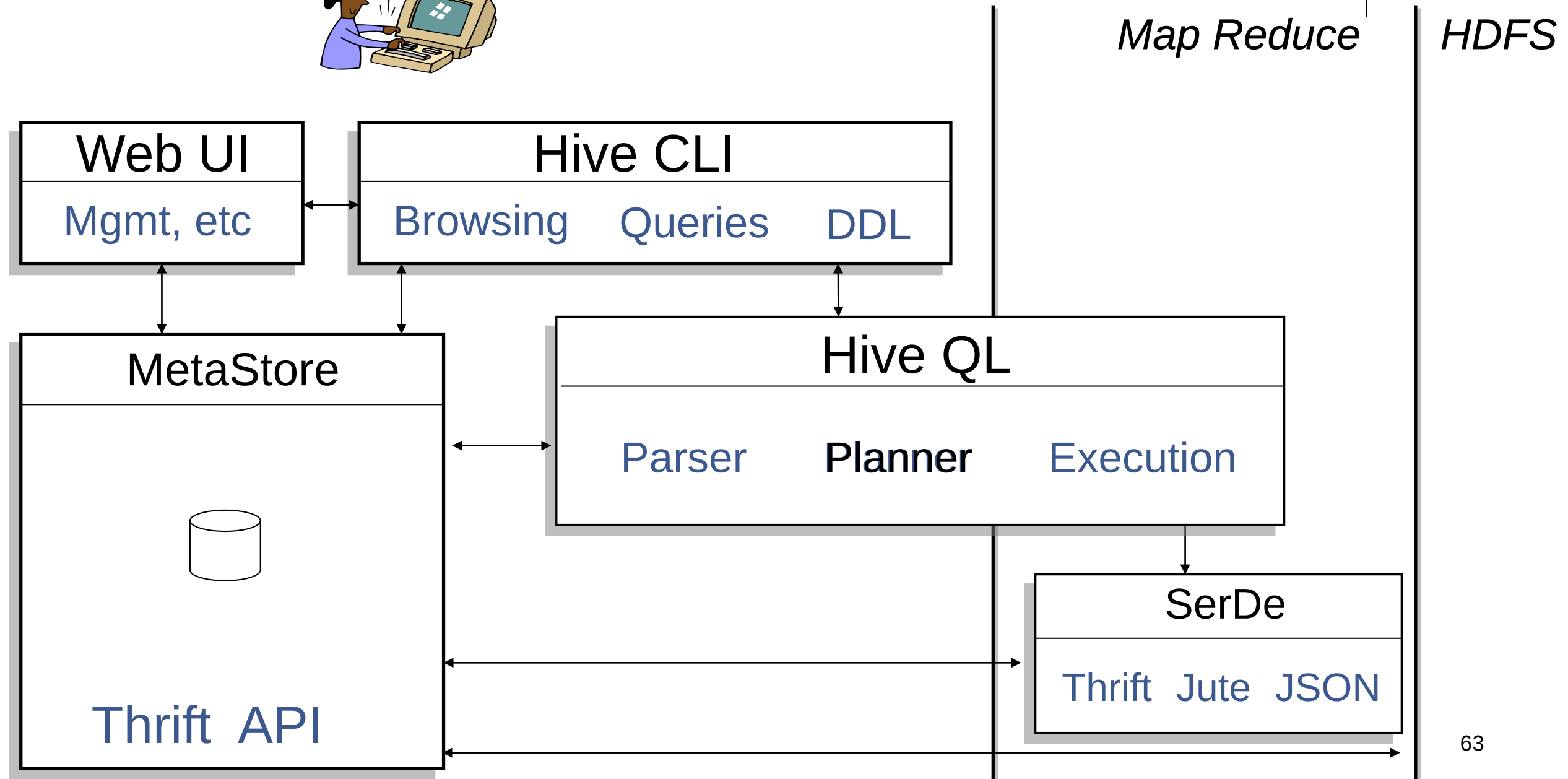
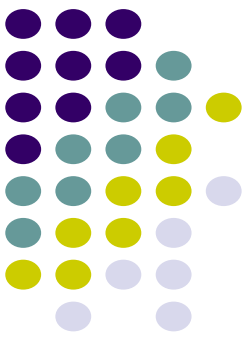
- **Map-Reduce is scalable**
 - SQL has a huge user base
 - SQL is easy to code
- **Solution: Combine SQL and Map-Reduce**
 - Hive on top of Hadoop (open source)
 - Aster Data (proprietary)
 - Green Plum (proprietary)

Hive

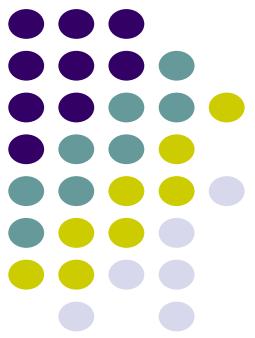


- **A database/data warehouse on top of Hadoop**
 - Rich data types (structs, lists and maps)
 - Efficient implementations of SQL filters, joins and group-by's on top of map reduce
- **Allow users to access Hive data without using Hive**
- **Link:**
 - <http://svn.apache.org/repos/asf/hadoop/hive/trunk/>

Hive Architecture



Hive QL – Join



page_view

page id	user id	time
1	111	9:08:01
2	111	9:08:13
1	222	9:08:14

X

user

user id	age	gender
111	25	female
222	32	male

=

pv_users

page id	age
1	25
2	25
1	32

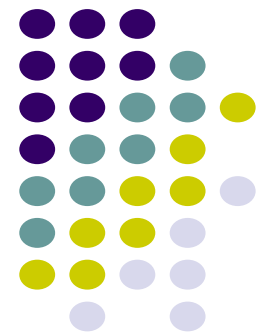
SQL:

```

INSERT INTO TABLE pv_users
SELECT pv.pageid, u.age
FROM page_view pv JOIN user u ON (pv.userid = u.userid);
    
```

FROM page_view pv JOIN user u ON (pv.userid = u.userid);

Hive QL – Join in Map Reduce



page_view

page id	user id	time
1	111	9:08:01
2	111	9:08:13
1	222	9:08:14
111	25	female
222	32	male

Map

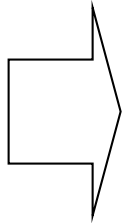
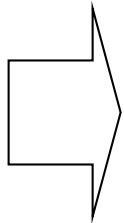
key	value
111	<1,1>
111	<1,2>
222	<1,1>

key	value
111	<2,25>
222	<2,32>

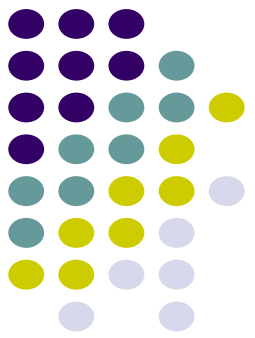
Shuffle Sort

key	value
111	<1,1>
111	<1,2>
111	<2,25>

key	value
222	<1,1>
222	<2,32>

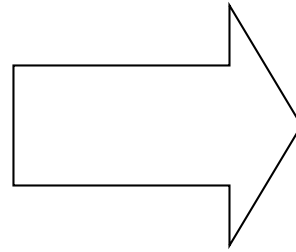


Hive QL – Group By



pv_users

page id	age
1	25
2	25
1	32
2	25



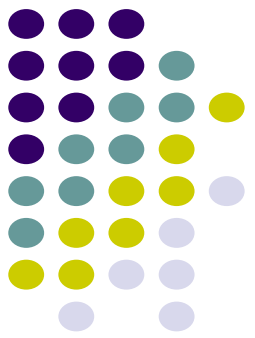
pageid_age_sum

page id	age	Count
1	25	1
2	25	2
1	32	1

- SQL:
 - INSERT INTO TABLE pageid_age_sum
 - SELECT pageid, age, count(1)
 - FROM pv_users
 - GROUP BY pageid, age;

Hive QL – Group By in Map

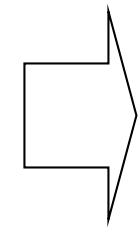
Reduce



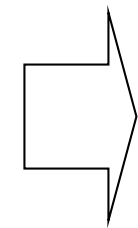
pv_users

page id	age
1	25
2	25

page id	age
1	32
2	25



Map



key	value
<1,2 5>	1
<2,2 5>	1

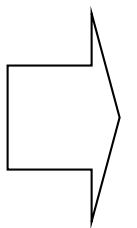
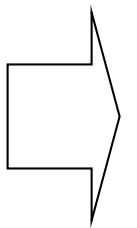
key	value
<1,3 2>	1
<2,2 5>	1

Shuffle
Sort

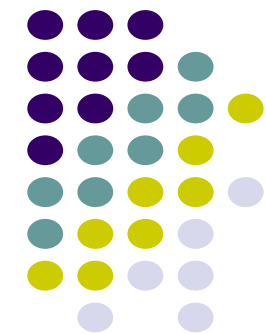
key	value
<1,2 5>	1
<1,3 2>	1

key	value
<2,2 5>	1
<2,2 5>	1

Reduce

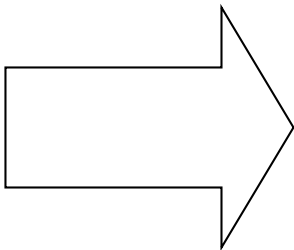


Hive QL – Group By with Distinct



page_view

page id	userid	time
1	111	9:08:01
2	111	9:08:13
1	222	9:08:14
2	111	9:08:20



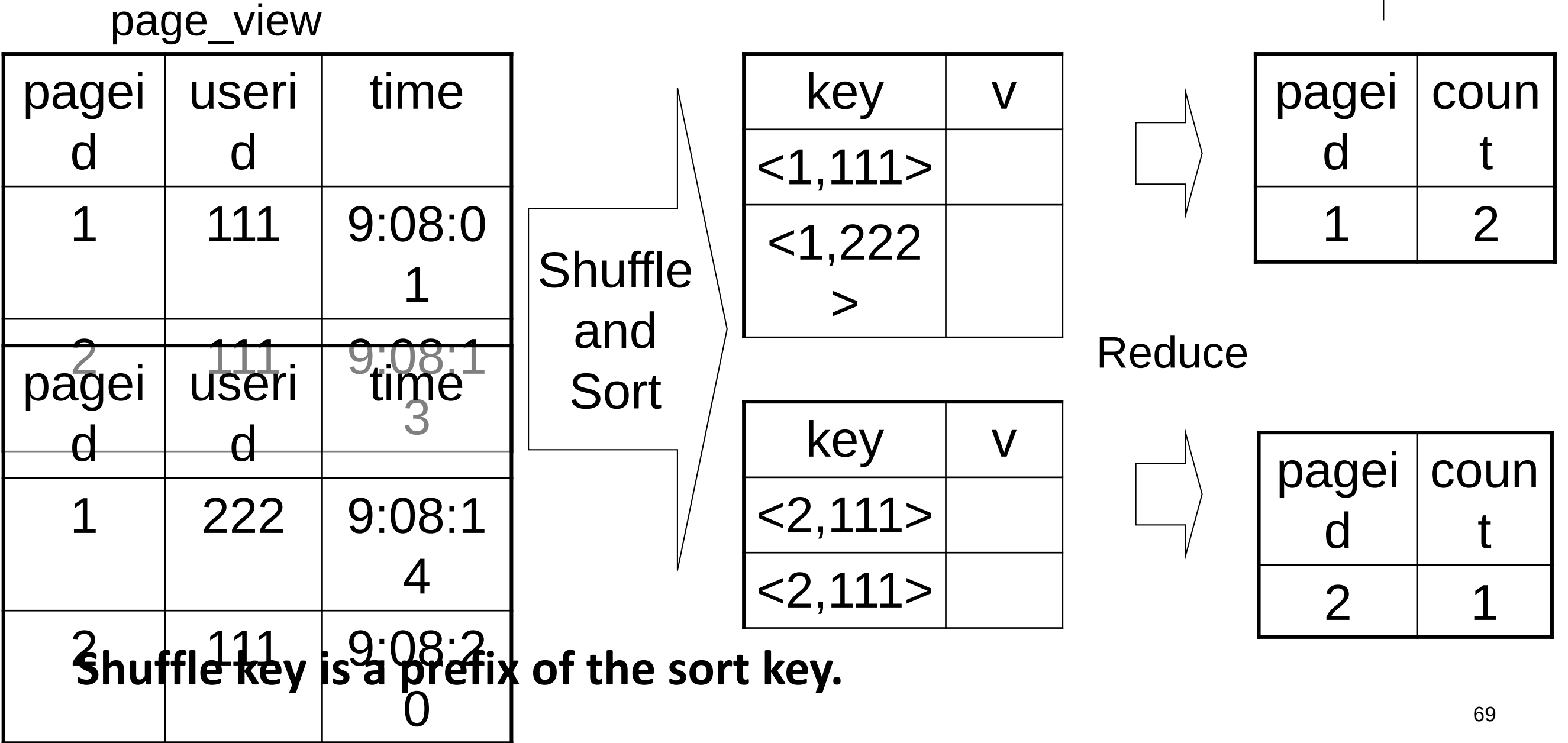
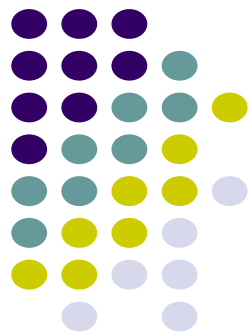
result

pageid	count_distinct_userid
1	2
2	1

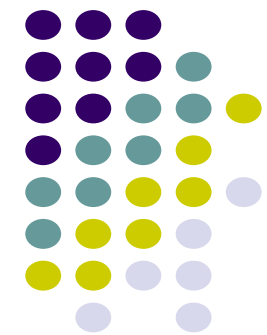
SQL

```
SELECT pageid, COUNT(DISTINCT userid)
FROM page_view
GROUP BY pageid
```

Hive QL – Group By with Distinct in Map Reduce



Hive QL: Order By

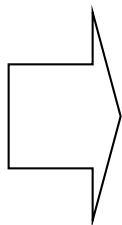
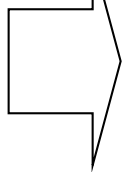


page_view

pageid	userid	time
2	111	9:08:13
1	111	9:08:01
2	111	9:08:20
1	222	9:08:14

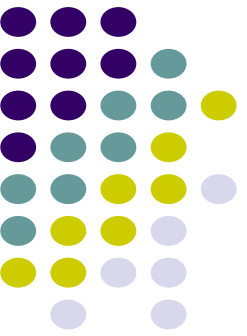
Shuffle and Sort

key	v
<1,111>	9:08:01
<2,111>	9:08:13
<1,222>	9:08:14
<2,111>	9:08:20



Reduce

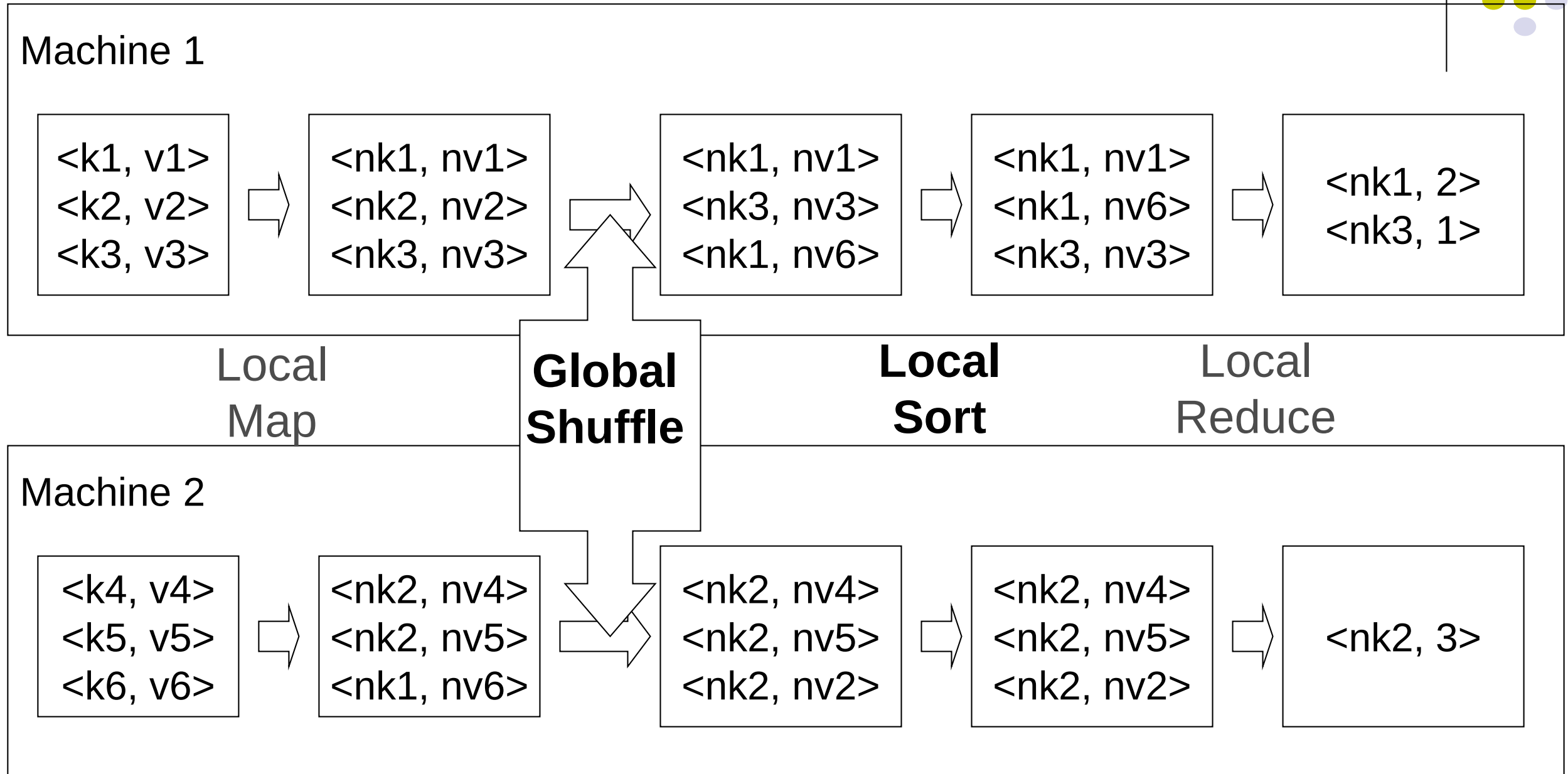
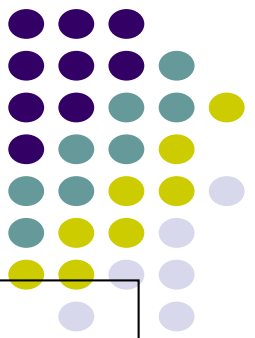
pageid	userid	time
1	111	9:08:01
2	111	9:08:13
1	222	9:08:14
2	111	9:08:20



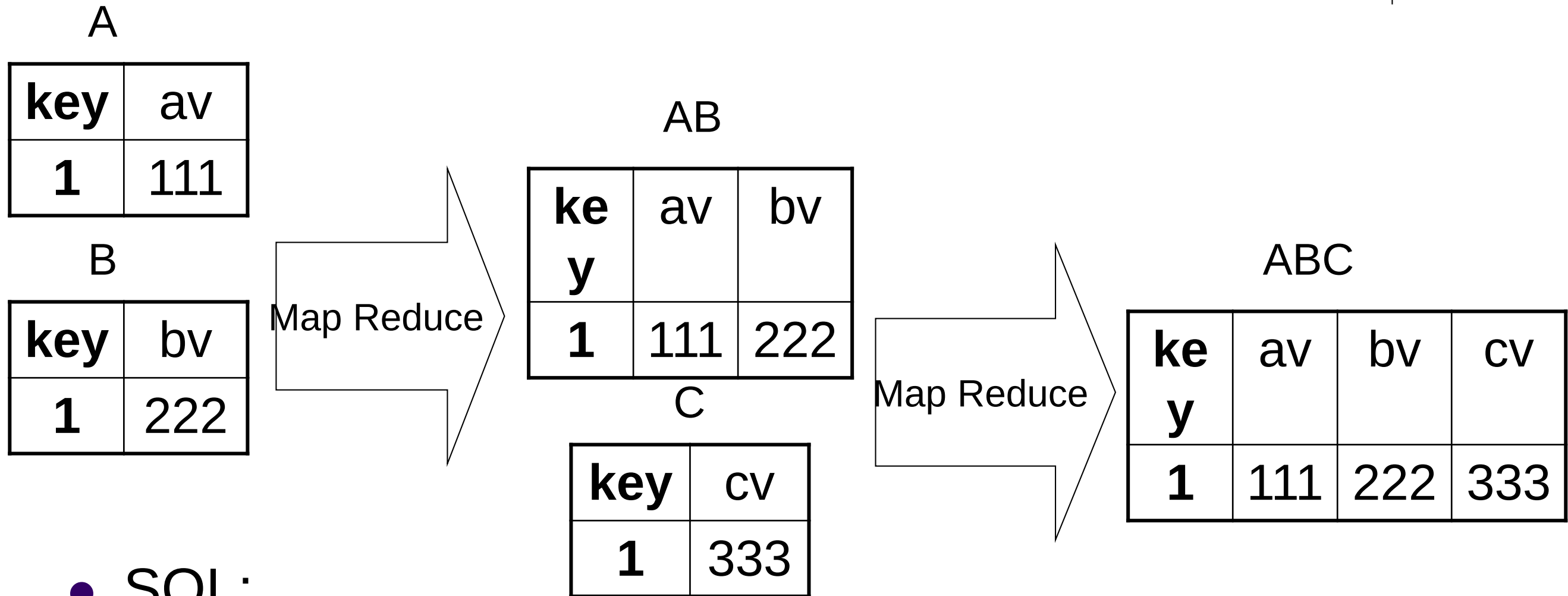
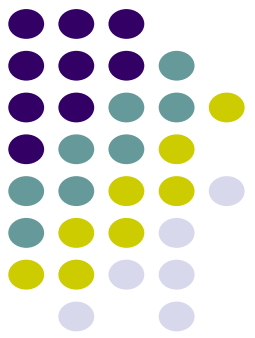
Hive Optimizations

Efficient Execution of SQL on top of Map-Reduce

(Simplified) Map Reduce Revisit



Merge Sequential Map Reduce Jobs

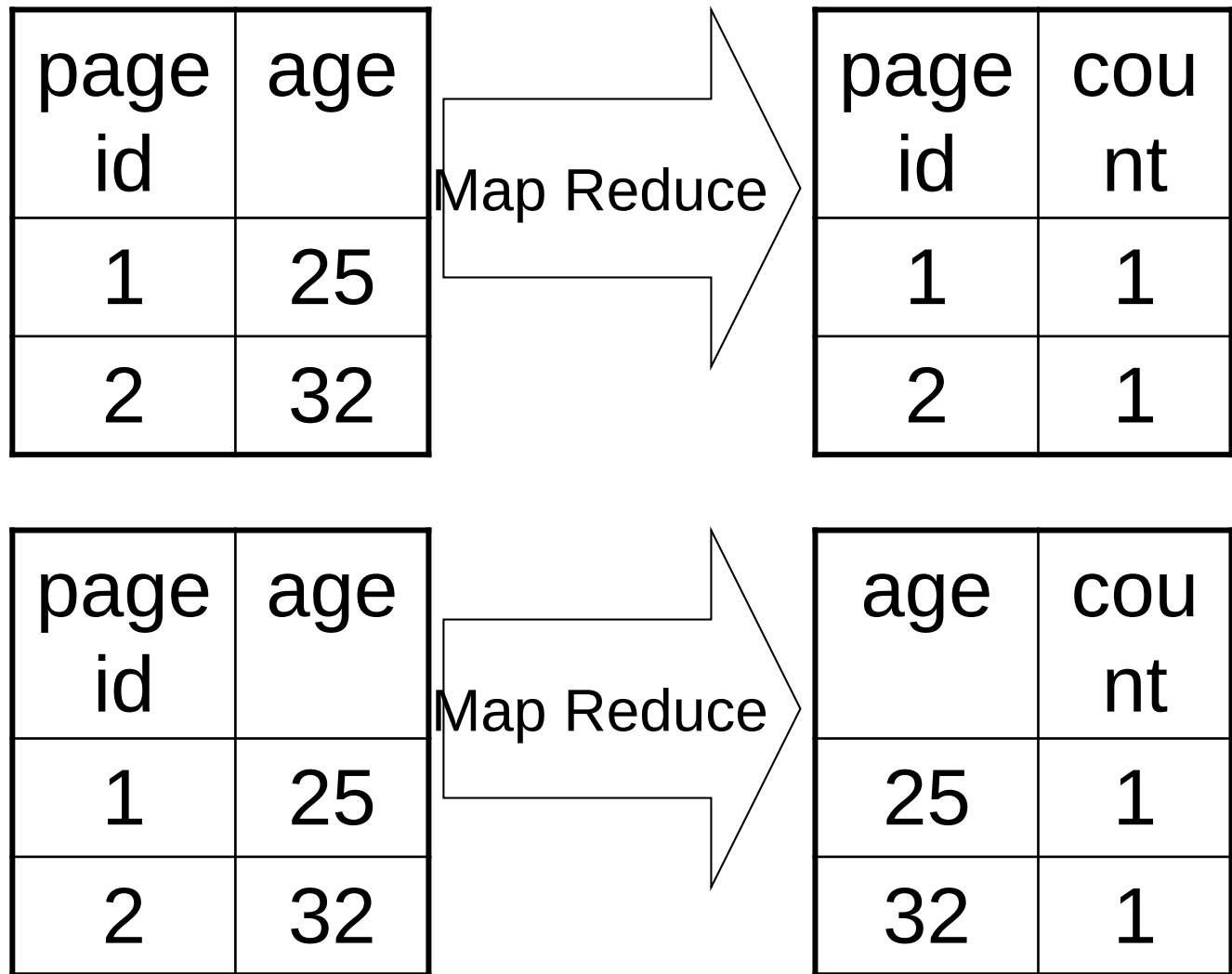
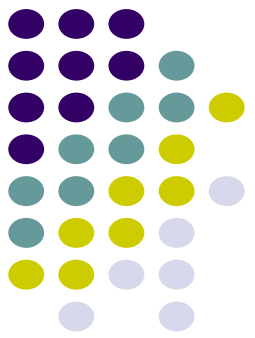


- SQL:

- FROM (a join b on a.key = b.key) join c on a.key = c.key SELECT

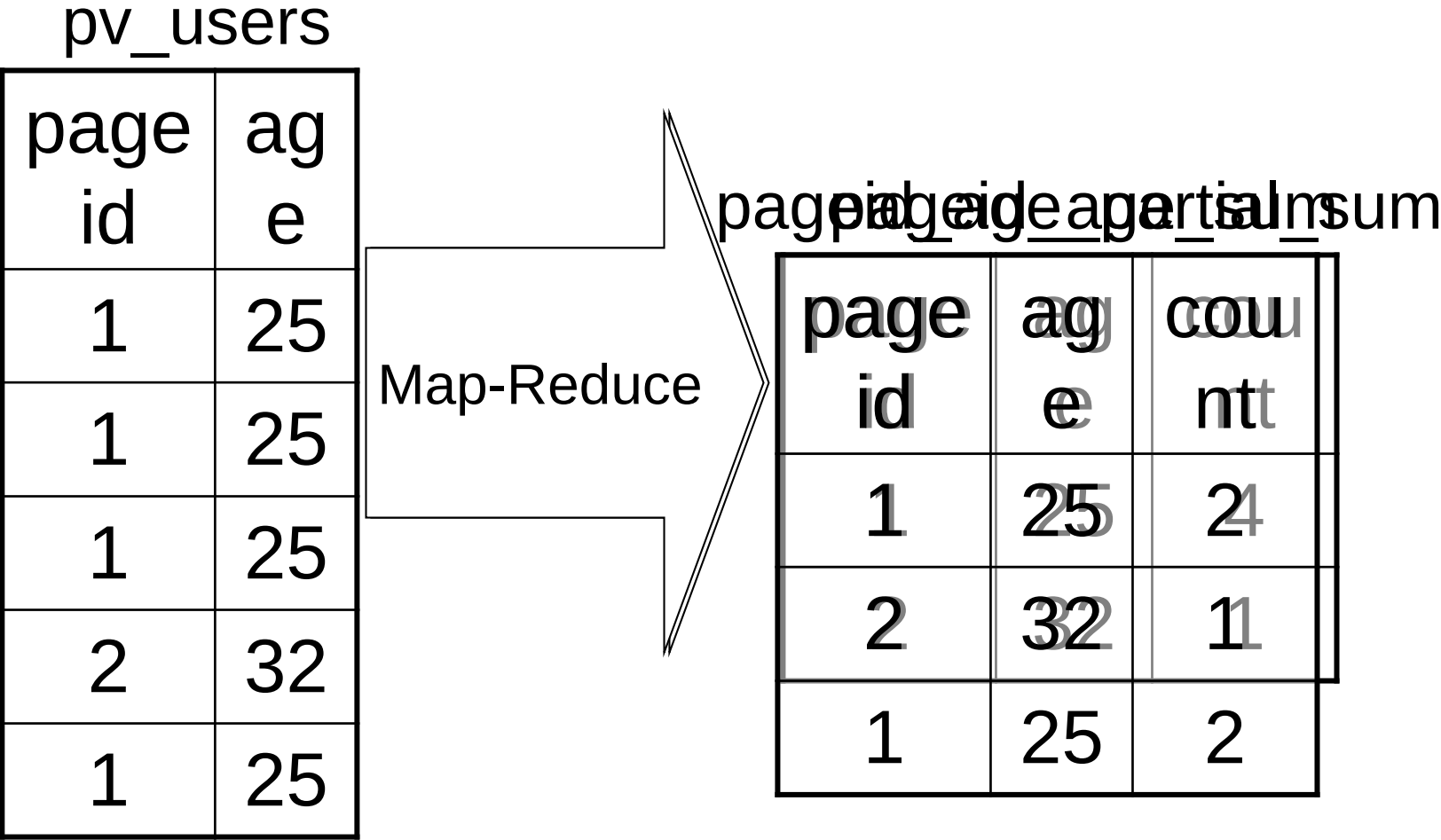
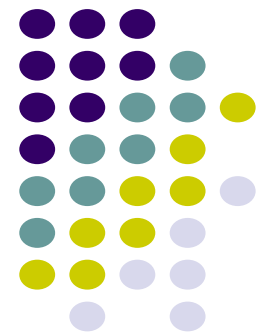
...

Share Common Read Operations

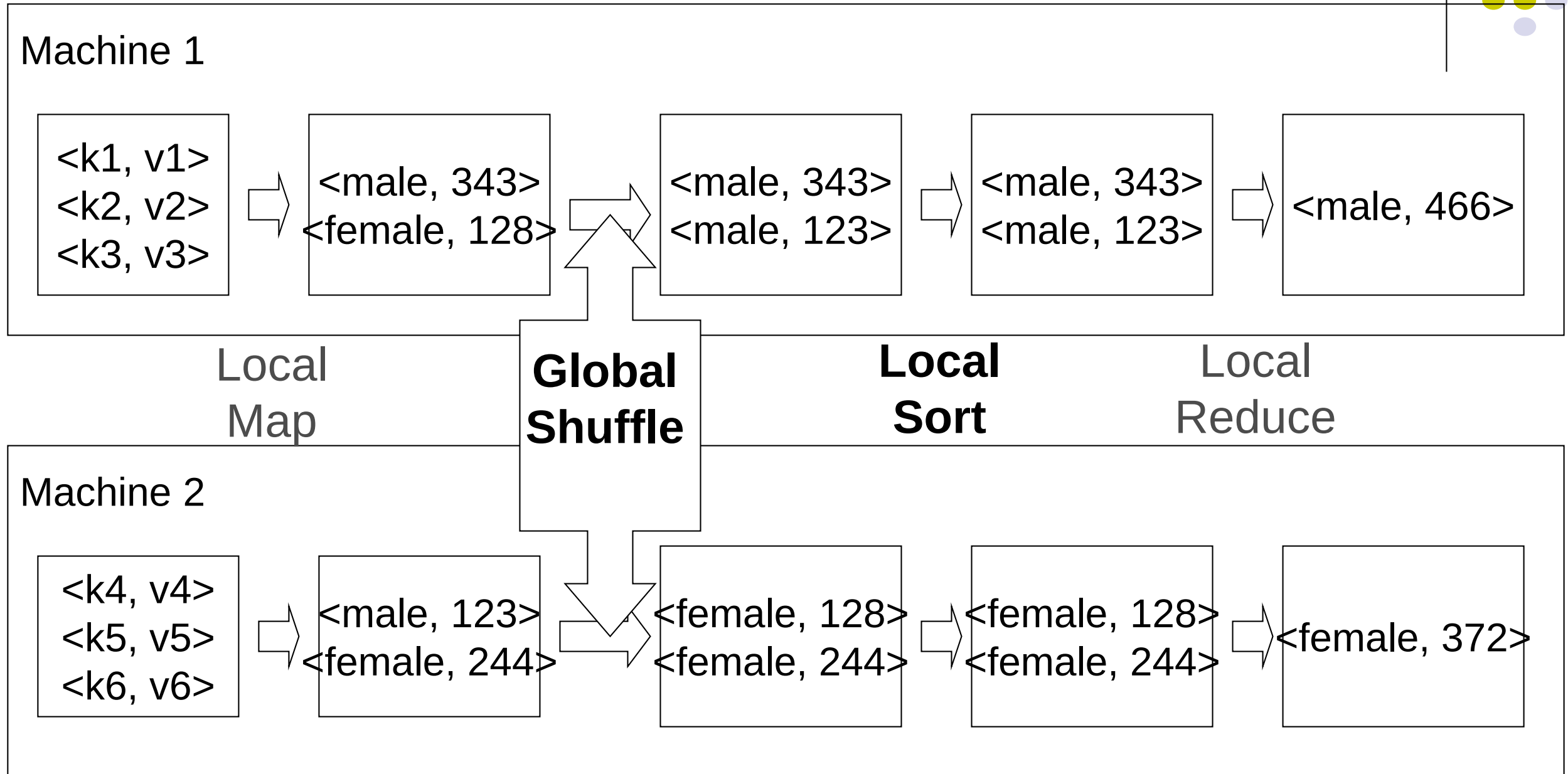
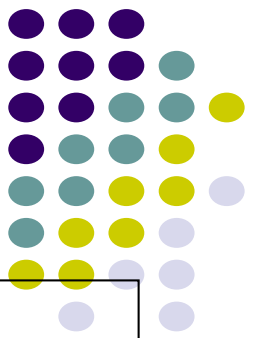


- Extended SQL
 - FROM pv_users
 - INSERT INTO TABLE pv_pageid_sum
 - SELECT pageid, count(1)
 - GROUP BY pageid
 - INSERT INTO TABLE pv_age_sum
 - SELECT age, count(1)
 - GROUP BY age;

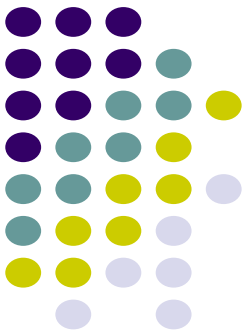
Load Balance Problem



Map-side Aggregation / Combiner



Query Rewrite



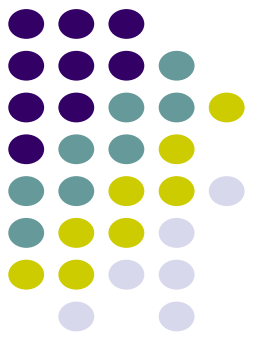
- **Predicate Push-down**

- select * from (select * from t) where col1 = '2008';

- **Column Pruning**

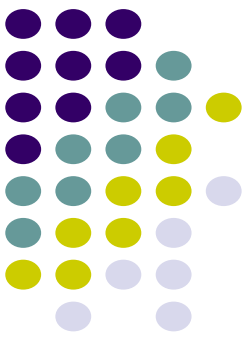
- select col1, col3 from (select * from t);

TODO: Column-based Storage and Map-side Join



url	page quality	IP
http://a.com/	90	65.1.2.3
http://b.com/	20	68.9.0.81
http://c.com/	68	11.3.85.1

url	clicked	viewed
http://a.com/	12	145
http://b.com/	45	383
http://c.com/	23	67



Credits

Presentations about Hadoop from [ApacheCon](#)
[Data Management at Facebook](#), *Jeff Hammerbacher*
[Hive – Data Warehousing & Analytics on Hadoop](#), *Joydeep*

Sen Sarma, Ashish Thusoo

People (for Hive project)

Suresh Anthony

Prasad Chakka

Namit Jain

Hao Liu

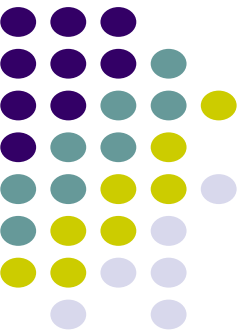
Raghu Murthy

Zheng Shao

Joydeep Sen Sarma

Ashish Thusoo

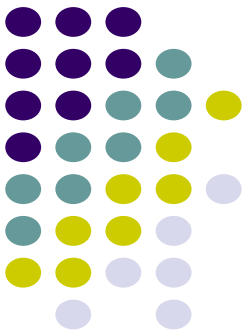
Pete Wyckoff



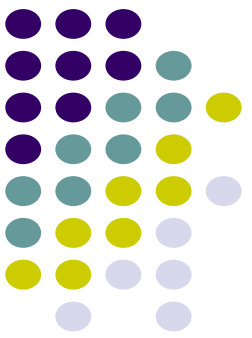
Questions?

zshao@apache.org

Appendix Pages

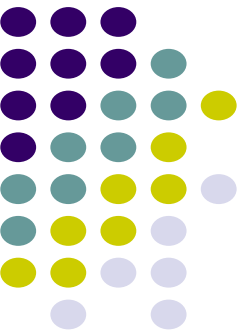


Dealing with Structured Data



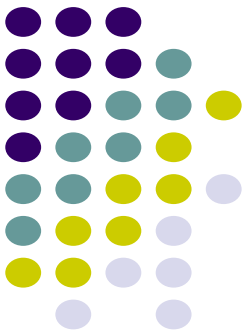
- Type system
 - Primitive types
 - Recursively build up using Composition/Maps/Lists
- Generic (De)Serialization Interface (SerDe)
 - To recursively list schema
 - To recursively access fields within a row object
- Serialization families implement interface
 - Thrift DDL based SerDe
 - Delimited text based SerDe
 - You can write your own SerDe
- Schema Evolution

MetaStore



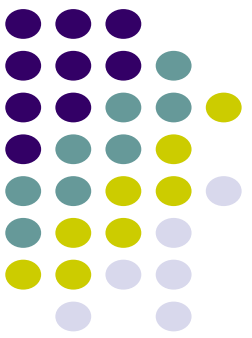
- Stores Table/Partition properties:
 - Table schema and SerDe library
 - Table Location on HDFS
 - Logical Partitioning keys and types
 - Other information
- Thrift API
 - Current clients in Php (Web Interface), Python (old CLI), Java (Query Engine and CLI), Perl (Tests)
- Metadata can be stored as text files or even in a SQL backend

Hive CLI



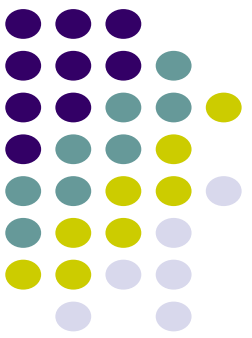
- DDL:
 - create table/drop table/rename table
 - alter table add column
- Browsing:
 - show tables
 - describe table
 - cat table
- Loading Data
- Queries

Web UI for Hive



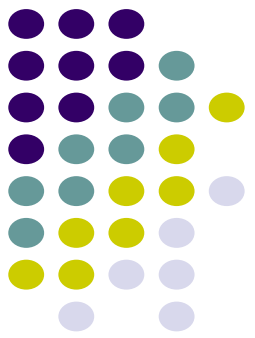
- MetaStore UI:
 - Browse and navigate all tables in the system
 - Comment on each table and each column
 - Also captures data dependencies
- HiPal:
 - Interactively construct SQL queries by mouse clicks
 - Support projection, filtering, group by and joining
 - Also support

Hive Query Language



- Philosophy
 - SQL
 - Map-Reduce with custom scripts (hadoop streaming)
- Query Operators
 - Projections
 - Equi-joins
 - Group by
 - Sampling
 - Order By

Hive QL – Custom Map/Reduce Scripts



- Extended SQL:
 - FROM (
 - FROM pv_users
 - **MAP** pv_users.userid, pv_users.date
 - **USING** 'map_script' AS (dt, uid)
 - **CLUSTER BY** dt) map
 - INSERT INTO TABLE pv_users_reduced
 - **REDUCE** map.dt, map.uid
 - **USING** 'reduce_script' AS (date, count);
- Map-Reduce: similar to hadoop streaming